

MOHSIN IBRAHIM HOSSAIN
AIUD, Algorithm Notes.

ALGORITHMS:-

~~LINEAR~~ ~~SEARCH~~

Linear Search:-

0	1	2	3	4	5	6	7
15	5	20	35	2	42	67	17

Let, the element to be searched is $K=42$

Simulation:-

15	5	20	35	2	42	67	17
----	---	----	----	---	----	----	----

↑
 $K \neq 15$

15	5	20	35	2	42	67	17
----	---	----	----	---	----	----	----

↑
 $K \neq 5$

15	5	20	35	2	42	67	17
----	---	----	----	---	----	----	----

↑
 $K \neq 20$

15	5	20	35	2	42	67	17
----	---	----	----	---	----	----	----

↑
 $K \neq 35$

15	5	20	35	2	42	67	17
----	---	----	----	---	----	----	----

↑
 $K \neq 2$

15	5	20	35	2	42	67	17
----	---	----	----	---	----	----	----

$K = 42$

Algorithm of Linear Search:-

linearSearch(a[], k){

for (i=0; i < array.length; i++){

if (a[i] == k){

print("Element found");

else{

print("Element Not found");

}

Time Complexity:-

1. Best Case $\rightarrow O(1)$

2. Average Case $\rightarrow O(n)$

3. Worst Case $\rightarrow O(n)$

Sun Mon Tue Wed Thu Fri Sat

Date

#BINARY
MEMO EARTH



Binary Search:-

0	1	2	3	4	5	6	7	8
10	12	24	29	39	40	51	56	69

Let, the element to Search is $K=56$.

Simulation:-

$beg = 0$; // Starting Index of array

$end = 8$; // Ending Index of array

$$mid = (beg + end) / 2; = (0 + 8) / 2 = 4$$

0	1	2	3	4	5	6	7	8
10	12	24	29	39	40	51	56	69



$$A[mid] \neq K \quad (39 \neq 56)$$

$$A[mid] < K \quad (39 < 56)$$

So, $beg = mid + 1 = 4 + 1 = 5$;

$$End = 8;$$

Now, $mid = (5 + 8) / 2 = 6$

0	1	2	3	4	5	6	7	8
10	12	24	29	39	40	51	56	69



$$A[mid] \neq K \quad (51 \neq 56)$$

$$A[mid] < K \quad (51 < 56)$$

So, $beg = mid + 1 = 6 + 1 = 7$;

$$End = 8;$$

Now, $mid = (8 + 7) / 2 = 7$



Date

D	D	M	M	Y	Y	Y	Y
---	---	---	---	---	---	---	---

Sun	Mon	Tue	Wed	Thu
-----	-----	-----	-----	-----

0	1	2	3	4	5	6	7	8
10	12	24	20	30	40	51	56	69

 $A[mid] == k$
 $(56 = 56)$

$\therefore$ Element found

What If $(A[mid] > k)$

if $A[mid] \neq k$

Also $A[mid] > k$

so $beg = beg;$

End = $mid - 1;$

Then! $mid = (beg + End) / 2;$

TIME Complexity:—

1. Best Case $\rightarrow O(1)$

2. Worst Case $\rightarrow O(\log n)$



ALGORITHM of BINARY SEARCH:-

BinarySearch(A, n, k)

Beg = 0;

End = n - 1; // n = Array Size.

mid = (Beg + End) / 2;

while (Beg < End)

if (A[mid] == k)

print("Data is found");

else if (A[mid] < k)

Beg = mid + 1;

else if (A[mid] > k)

End = mid - 1;



Date

D	D	M	M	Y	Y	Y	Y
---	---	---	---	---	---	---	---

Sun Mon Tu

#Insertion Sort

Insertion Sort:-

0	1	2	3	4
23	1	10	5	2

Initial array

Simulation:-

Step: 1:-

0	1	2	3	4	
23	1	10	5	2	

0	1	2	3	4	
23	23	10	5	2	1 temp

0	1	2	3	4	
1	23	10	5	2	

Step: 2:-

0	1	2	3	4	
1	23	10	5	2	

0	1	2	3	4	
1	23	23	5	2	10 temp

0	1	2	3	4	
1	10	23	5	2	

Step:3:-

1	10	23	5	2
---	----	----	---	---

1	10	23	23	2
---	----	----	----	---

5 temp

1	10	10	23	2
---	----	----	----	---

1	5	10	23	2
---	---	----	----	---

Step:4:-

1	5	10	23	2
---	---	----	----	---

1	5	10	23	23
---	---	----	----	----

2 temp

1	5	10	10	23
---	---	----	----	----

1	5	5	10	23
---	---	---	----	----

1	2	5	10	23
---	---	---	----	----



Insertion Sort Algorithm:-

insertionSort(A[], n){

for($i=1$; $i \leq n$; $i++$)

temp = A[i];

$j = i - 1$;

while($j > 0$ && $A[j] > temp$)

$A[j+1] = A[j]$;

$j--$;

$A[j+1] = temp$;

}

Time Complexity:-

1. Best Case $\rightarrow O(n)$

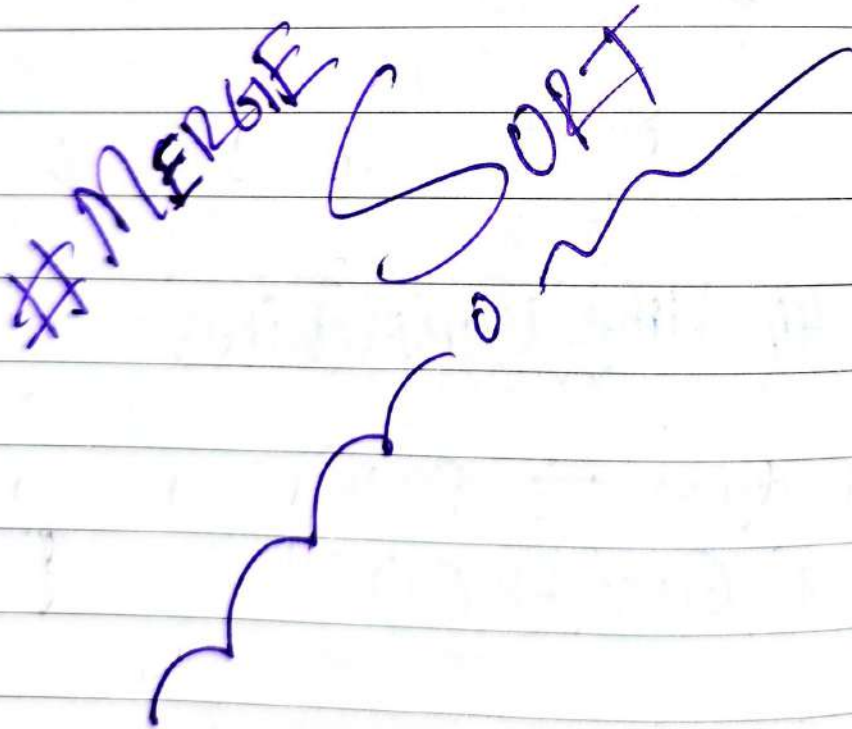
2. Worst Case $\rightarrow O(n^2)$

Date

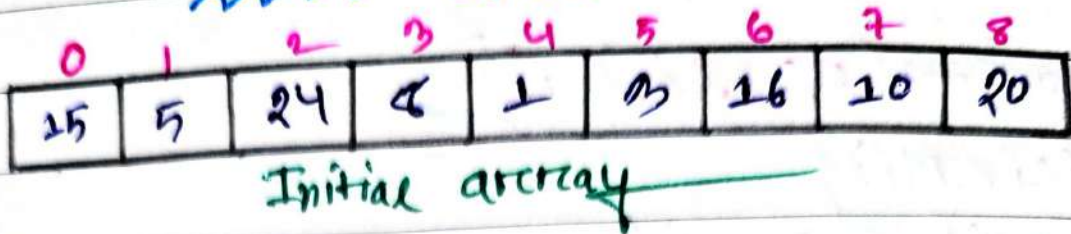
D	D	M	M	Y	Y	Y	Y
---	---	---	---	---	---	---	---

Sun	Mon	Tue	Wed
-----	-----	-----	-----

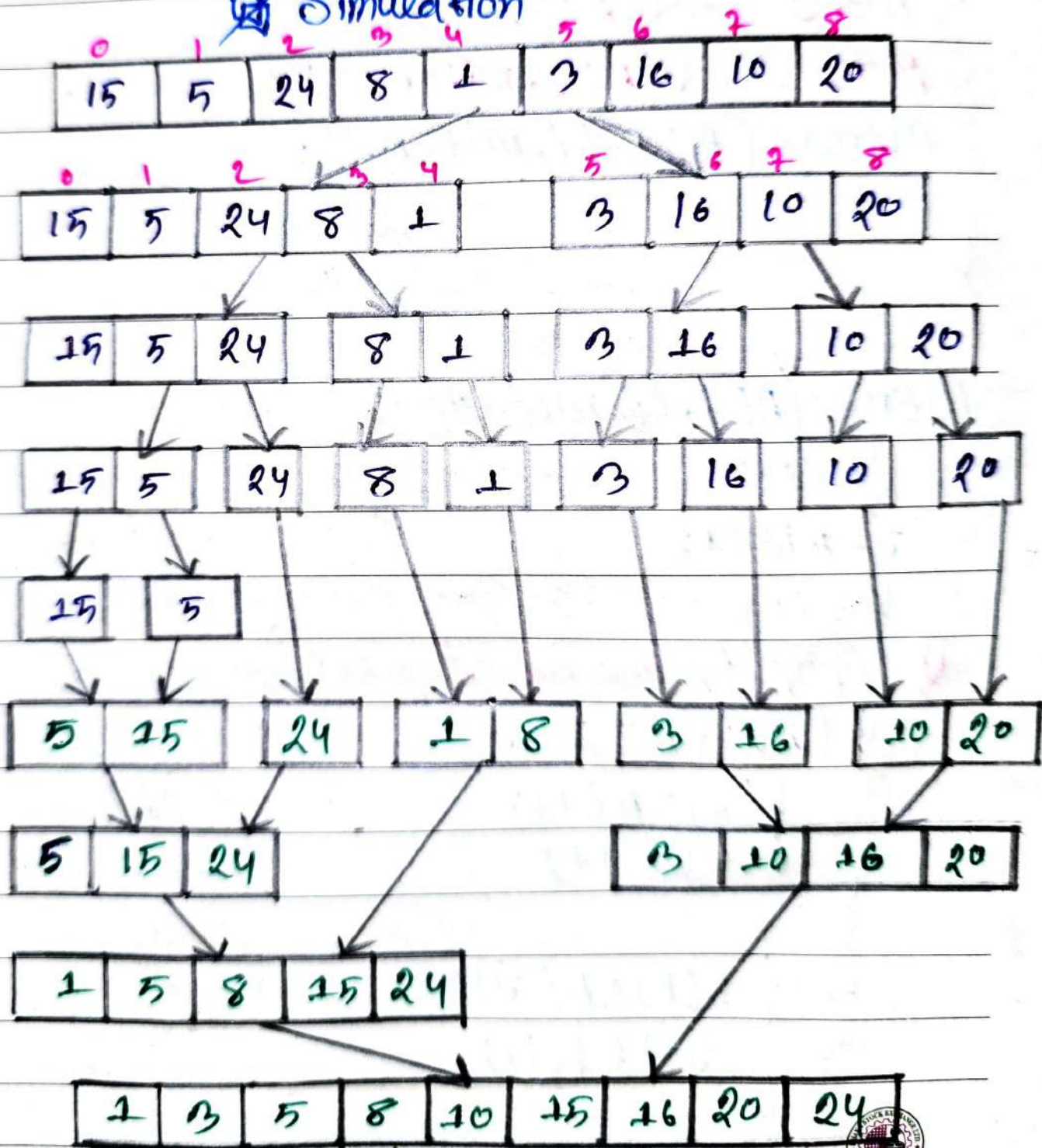
#MERGE SORT



MERGE SORT:-



Simulation



Merge Sort Algorithm:-

```

MergeSort(A[1], Li, Mi) {
    if (Li < Mi) {
        mid = (Li + Mi) / 2;
        MergeSort(A[1], Li, mid);
        MergeSort(A[1], mid + 1, Mi);
        Merge(A[1], Li, mid, Mi);
    }
}

```

```

Merge(A[1], Li, mid, Mi) {

```

```

    i = Li;

```

```

    j = mid + 1;

```

```

    k = Li;

```

```

    while (i <= mid && j <= Mi) {

```

```

        if (A[i] <= A[j]) {

```

```

            B[k] = A[i];

```

```

            i++; k++;

```

```

        }

```

```

    else { B[k] = A[j];

```

```

        j++; k++;

```



```

while( i <= mid ) {
    B[k] = A[i]
    i++; k++;
}

```

```

while( j <= mid ) {
    B[k] = A[j]
    k++; j++;
}

```

Time Complexity:-

1. Best Case $\rightarrow O(n \log n)$

2. Worst Case $\rightarrow O(n \log n)$

Date

D	D	M	M	Y	Y	Y	Y
---	---	---	---	---	---	---	---

Sun	Mon	Tue	W
-----	-----	-----	---

Quick Sort

#Quick Sort:-

7 6 10 5 0 2 1 15 7

Initial Array

Simulation:-

Pivot → 7 Start → 7 End → 7

Pivot → 7 Start → 10 End → 7

7 > 7 7 > 6 7 < 10

Pivot → 7 Start → 10 End → 7

Pivot → 7 Start → 10 End → 7

Pivot → 7 Start → 10 End → 7

7 > 7 7 > 5 7 < 0

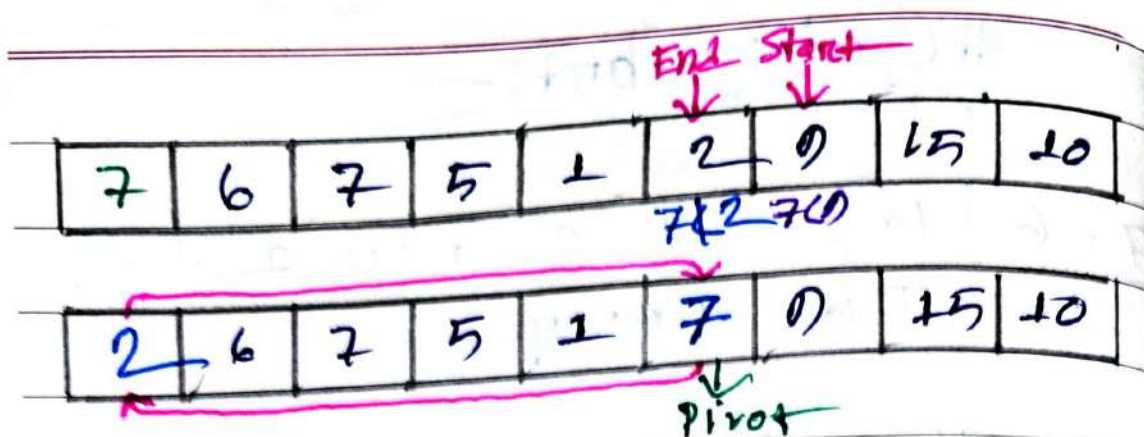
Pivot → 7 Start → 10 End → 7

7 < 1 7 < 5 7 < 10

Pivot → 7 Start → 10 End → 7

Pivot → 7 Start → 10 End → 7

7 < 1 7 > 2 7 < 0



Here, we will get two parts to continue the Quick Sort.

So, rest of the part will follow the above procedure again & finally we will get the following sorted array.



TIME Complexity—

1. Best Case $\rightarrow O(n \log n)$

2. Worst Case $\rightarrow O(n^2)$

QuickSort Algorithm:-

```

QuickSort(A[ ], Li, Mi) {
    if (Li < Mi) {
        Loc = Partition(A[ ], Li, Mi);
        QuickSort(A[ ], Li, Loc-1);
        QuickSort(A[ ], Loc+1, Mi);
    }
}

```

Partition(A[], Li, Mi)

Pivot = A[Li]

Start = Li

End = Mi

While (Start < End) {

While (A[Start] ≤ Pivot) {

Start++;

While (A[End] > Pivot) {

End--;

if (Start < End) {

Swap(A[Start], A[End]);

Swap(A[Li], A[End]);

return End;



~~Counting~~ Sort



Counting Sort:-

0	1	2	3	4	5	6	7	8	9
2	1	1	0	2	5	4	6	2	8
7	7	0	2	0	1	0			
10	11	12	13	14	15	16			

$$n = 17$$

$$k = 0$$

Simulation:-

Count =

0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

Count =

0	1	2	3	4	5	6	7	8	9
3	3	4	0	1	1	0	2	1	2

Count =

0	1	2	3	4	5	6	7	8	9
3	6	10	10	11	12	12	14	15	17

0	1	2	3	4	5	6	7	8	9
0	0	0	1	1	1	2	2	2	2
4	5	7	7	8	0	0			

Counting Sort Algorithm:-

CountSort(A[], n, k)

Count[] = Count[k+1]

B[] = B[k]

for (i = 0; i < n; i++)
++ Count[A[i]];

for (i = 1; i <= n; i++)
Count[i] = Count[i] + Count[i-1];

for (i = n-1; i >= 0; i--)
B[--Count[A[i]]] = A[i];

for (i = 0; i < n; i++)
A[i] = B[i]

Time Complexity:-

1. Best case $\rightarrow O(n + \max)$

2. Worst case $\rightarrow O(n + \max)$



Selection Sort

Selection Sort:-

0	1	2	3	4	5
7	4	10	8	3	1

Initial Array

Simulation:-

7	4	10	8	3	1
---	---	----	---	---	---

1	4	10	8	3	7
---	---	----	---	---	---

1	3	10	8	4	7
---	---	----	---	---	---

1	3	4	8	10	7
---	---	---	---	----	---

1	3	4	7	10	8
---	---	---	---	----	---

1	3	4	7	8	10
---	---	---	---	---	----

Selection Sort Algorithm:-

```
selectionSort(A, n){  
  for(i=0; i < n-1; i++){  
    int min = i;  
    for(j=i+1; j < n; j++){  
      if(A[j] < A[min]){  
        min = j;  
      }  
    }  
    if(min != i){  
      swap(A[i], A[min]);  
    }  
  }  
}
```

Time Complexity

1. Best Case $O(n^2)$

2. Average Case $O(n^2)$

3. Worst Case $O(n^2)$

Bubble Sort

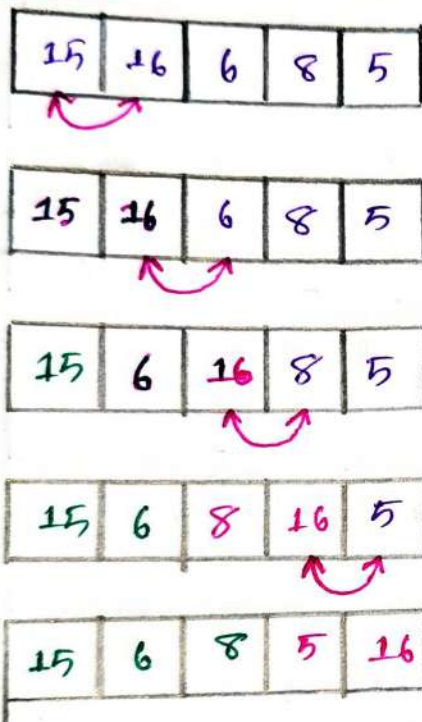
Bubble Sort

0	1	2	3	4
15	16	6	8	5

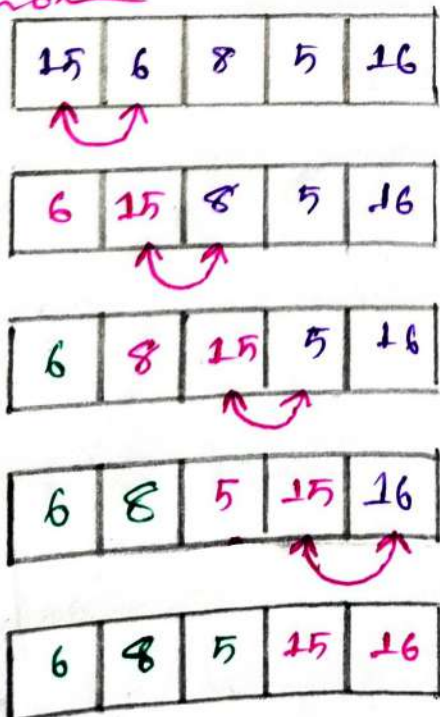
Initial Array.

Simulation:-

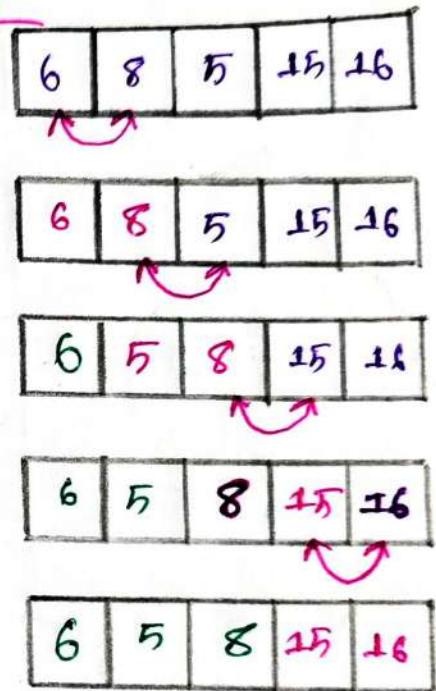
Pass 1:-



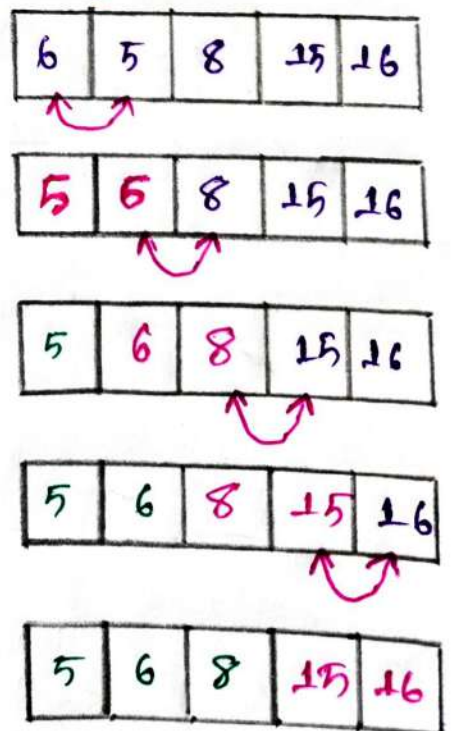
Pass 2:-



Pass 3:-



Pass 4:-



Bubble Sort Algorithm:-

```
BubbleSort(A[ ], n){  
  for(i=0; i < n-1; i++){  
    for(j=0; j < n-1; j++){  
      if(A[j] > A[j+1]){  
        temp = A[j];  
        A[j] = A[j+1];  
        A[j+1] = temp;  
      }  
    }  
  }  
}
```

To avoid unnecessary comparisons, we can modify the Algorithm:-

```
BubbleSort(A[ ], n){  
  for(i=0; i < n-1; i++){  
    for(j=0; j < n-1-i; j++){  
      if(A[j] > A[j+1]){  
        temp = A[j];  
        A[j] = A[j+1];  
        A[j+1] = temp;  
      }  
    }  
  }  
}
```

Time Complexity:-

1. Best Case = $O(n)$
2. Worst Case = $O(n^2)$
3. Average Case = $O(n^2)$

Greedy Algorithm

The Coin Changing:-

Greedy Algorithm:-

⇒ Greedy algorithm make the choice that looks best at the moment.

Some of them are:-

1. The Coin Changing

2. Fractional knapsack

3. Huffman Coding

4. Bin packing

The Coin Changing:-

Simulation:-

problem: 1:-

if Amount $S = 1757$ &

Available Coins $D = \{1000, 500, 200, 100, 50, 20, 10, 5, 2, 1\}$

$C =$ Largest coin in D that is $\leq S$.

Step: 1:-

$$S = 1757$$

$$C = 1000$$

$$\text{num} = S/C = 1757/1000 \\ = 1$$

$$S = S - C * \text{num} \\ = 1757 - 1 * 1000 \\ = 757$$

Step 2:—
mon

$$S = 757$$

$$c = 500$$

$$\text{num} = S/c = 757/500$$
$$= 1$$

$$S = S - \text{num} * c$$

$$= 757 - 1 * 500$$

$$= 257$$

Step 3:—
mon

$$S = 257$$

$$c = 200$$

$$\text{num} = S/c = 257/200$$
$$= 1$$

$$S = S - \text{num} * c$$

$$= 257 - 1 * 200$$

$$= 57$$

Step 4:—
mon

$$S = 57$$

$$c = 50$$

$$\text{num} = S/c = 57/50$$
$$= 1$$

$$S = S - \text{num} * c$$

$$= 57 - 50 * 1$$

$$= 7$$

Step: 5:-

$$S = 7$$

$$C = 5$$

$$\text{num} = S/C = 7/5 \\ = 1$$

$$S = S - \text{num} * C$$

$$= 7 - 1 * 5$$

$$= 2$$

Step: 6:-

$$S = 2$$

$$C = 2$$

$$\text{num} = S/C = 2/2 \\ = 1$$

$$S = S - \text{num} * C$$

$$= 2 - 1 * 2$$

$$= 0$$

Stop. When $S = 0$

Result: 1000×1

500×1

200×1

50×1

5×1

2×1

Coin Changing Algorithm:-

CoinChange(s, D) {

// s = total amount

// D = list of coin

while($s > 0$) {

c = Largest coin in D that is $\leq s$;

int num = s/c ;

$s = s - c * \text{num}$;

}

Time Complexity:-

$\Rightarrow$ This is $\Theta(m)$

where, m = number of denominations.

Fractional Knapsack

Fractional Knapsack

⇒ It is an optimization problem where we maximize the total value of items in a knapsack with fixed capacity. Items can be divided into fraction.

Solving Strategy:-

Step: 1:-

⇒ Calculate the value per weight for each item.

$$\text{Value per unit weight} = \frac{V}{W}$$

Step: 2:-

⇒ Sort all item in descending order of their value per unit weight.

Step: 3:-

⇒ pick items greedily:-

1. Start with that item has the highest value per unit weight.

2. Take as much the item as possible, until the knapsack is full.

Step: 4:-

⇒ Stop when the knapsack is full.

problem: 1:—

⇒ Let, you have to bring 25kg of product and you have the following supply: 7kg rice, 5kg lentils, 2kg sugar, 5kg salt, 3kg basmati rice of 700 tk, 600 tk, 240 tk, 250 tk, 1050 tk. How will you get optimization to maximize your profit.

Fractional Knapsack

Simulation:—

Step 1:— Find Value per weight (v)

Item	Value (V)	weight	Value per weight (v)
Rice	700	7	100
Lentils	600	5	120
Sugar	240	2	120
Salt	250	5	50
Basmati	1050	3	350

Step 2:- Sort item based on value per weight in descending order

Item	Value (v)	Weight (w)	Value per weight (v)
Basmati	1050	3	350
Lentils	600	5	120
Sugar	240	2	120
Rice	700	7	100
Salt	250	5	50

Step 3:- pick items greedily

Page: 1:-

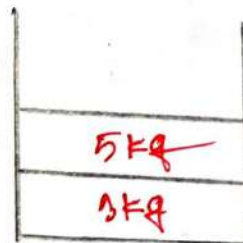


$$W = \text{Maximum capacity} = 15 \text{ kg}$$

$$\begin{aligned} w &= \text{remaining weight} \\ &= W - w_i \\ &= 15 - 3 \\ &= 12 \text{ kg} \end{aligned}$$

$$\begin{aligned} \text{profit} &= \text{Value} = \text{Value}(i) \\ \therefore \text{value} &= 1050 \end{aligned}$$

Page: 2:-



$$\begin{aligned} w &= \text{remaining weight} \\ &= 12 \text{ kg} - 5 \text{ kg} \\ &= 7 \text{ kg} \end{aligned}$$

$$\begin{aligned} \text{Value} &= \text{Value}(i) + \text{Value}(j) \\ &= 600 + 1050 \\ &= 1650 \end{aligned}$$

Step: 3:-
mon

1kg
5kg
3kg

$$W = W - w_i$$

$$= 7kg - 2kg$$

$$= 5kg$$

$$\text{Value}(V) = \text{Value}(V) + V_i$$

$$\therefore V = V + V_i$$

$$= 1650 + 240$$

$$= 1890$$

Step: 4:-
mon

5kg
2kg
5kg
3kg

Since $w_i = 7kg > W = 5$

So, take fraction $w_i = W$

$$w_i = W$$

$$\therefore W = W - w_i$$

$$= 5 - 5$$

$$= 0kg$$

$$\text{Value}(V) = V + v_i * w_i$$

$$= 1890 + 100 * 5$$

$$= 2390$$

Step: 4:-
mon

$\Rightarrow$ Stop [Since $w = 0$]

Time Complexity:

1. Best Case = $O(2^n)$

2. Average Case = $O(2^n)$

3. Worst Case = $O(2^n)$

Fractional knapsack Algorithm:-

Input:-

$S = \{(w_i, b_i)\}$ // set of items. = S
 $W = \text{Max Capacity of the knapsack}$ // $w_i = \text{items weight}$
// $b_i = \text{items value.}$

Output:-

$x_i = \text{weight for each item into the knapsack}$
 $P = \text{Total value of selected items}$

$\text{FractionalKnapsack}(S, W)$

$p \leftarrow 0;$

$w \leftarrow 0;$

$x[i] \leftarrow 0;$

$v[i] \leftarrow \frac{b[i]}{w[i]};$

Sort(S by $v[i]$ in descending order);

for each (item i in S)

if ($w + w[i] < W$)

$x[i] \leftarrow w[i];$

$p \leftarrow p + v[i];$

$w \leftarrow w + w[i];$

else

$x[i] \leftarrow W - w;$

$p \leftarrow p + v[i] \cdot x[i];$

$w \leftarrow W;$

Return $\{x[i]\}, p;$

#Huffman

Codes

Huffman Coding

Huffman Coding

- Huffman Coding is a famous Greedy Algorithm.
- It is used for the lossless compression of data.
- It uses variable length encoding.s
- It assigns variable length code to all the characters.
- The code length of a character depends on how frequently it occurs in the given text.
- The character which occurs most frequently gets the smallest code.
- The character which occurs least frequently gets the largest code.
- It is also known as **Huffman Encoding**.

Prefix Rule

- Huffman Coding implements a rule known as a prefix rule.
- This is to prevent the ambiguities while decoding.
- It ensures that the code assigned to any character is not a prefix of the code assigned to any other character.

Major Steps in Huffman Coding

⇒ **There are two major steps in Huffman Coding-**

1. Building a Huffman Tree from the input characters.
2. Assigning code to the characters by traversing the Huffman Tree.

Huffman Tree

Step-01:

- Create a leaf node for each character of the text.
- Leaf node of a character contains the occurring frequency of that character.

Step-02:

- Arrange all the nodes in increasing order of their frequency value.

Step-03:

⇒ Considering the first two nodes having minimum frequency,

- Create a new internal node.
- The frequency of this new node is the sum of frequency of those two nodes.
- Make the first node as a left child and the other node as a right child of the newly created node.

Step-04:

- Keep repeating Step-02 and Step-03 until all the nodes form a single tree.
- The tree finally obtained is the desired Huffman Tree.

PRACTICE PROBLEM BASED ON HUFFMAN CODING

⇒ A file contains the following characters with the frequencies as shown. If Huffman Coding is used for data compression, determine-

1. Huffman Code for each character

Characters	Frequencies
a	10
e	15
i	12
o	3
u	4
s	13
t	1

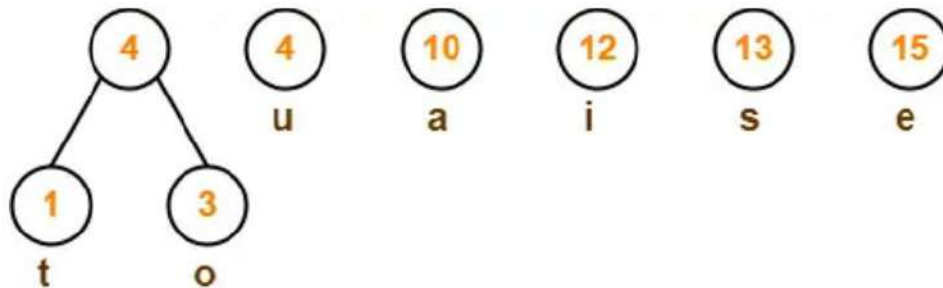
Solution:

⇒ First let us construct the Huffman Tree.

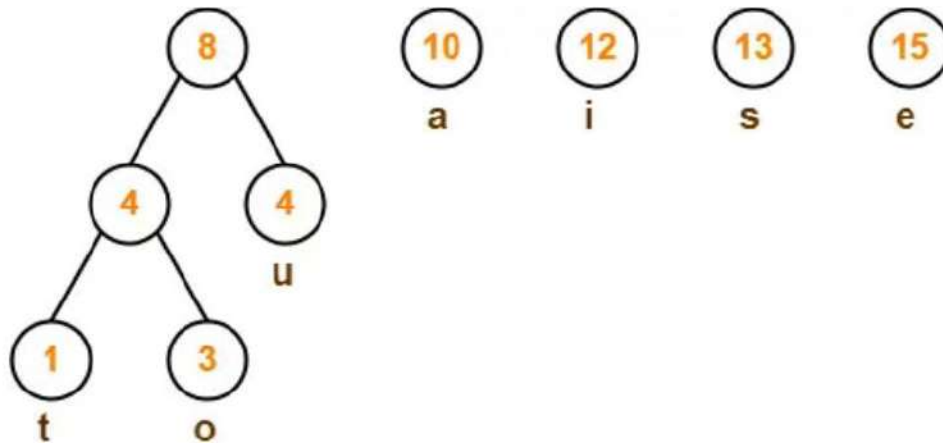
Step-01:



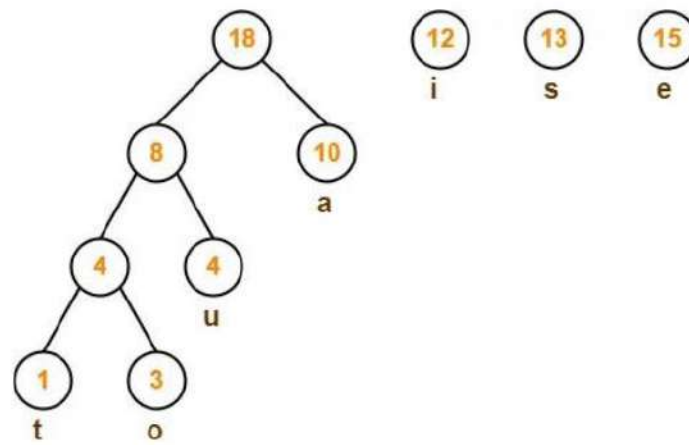
Step-02:



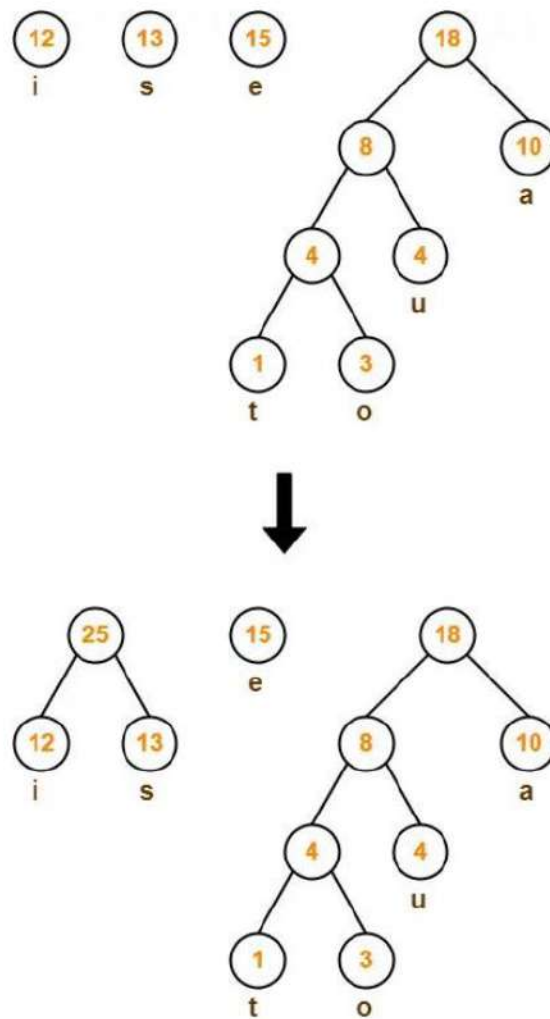
Step-03:



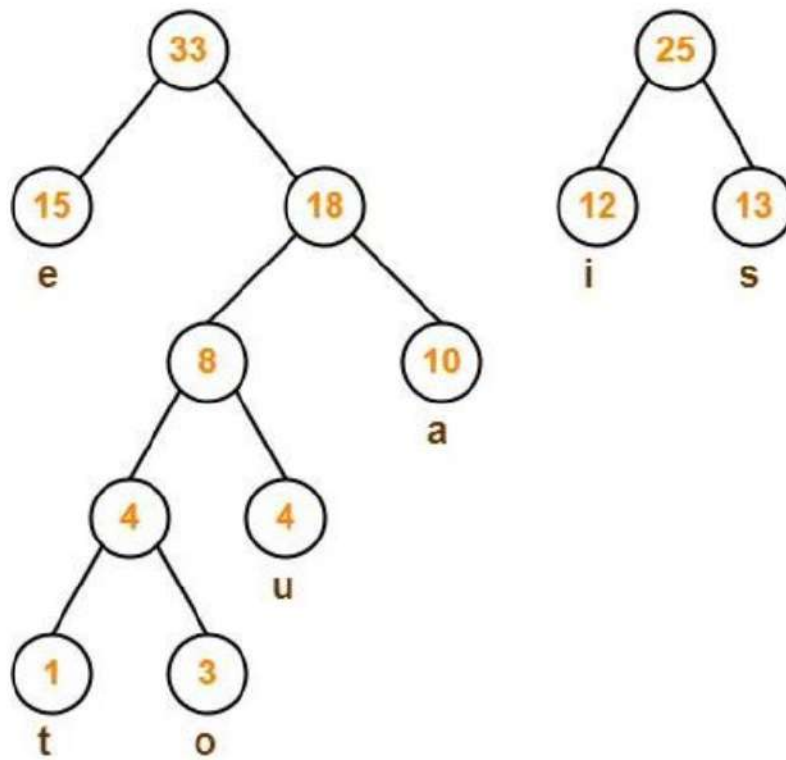
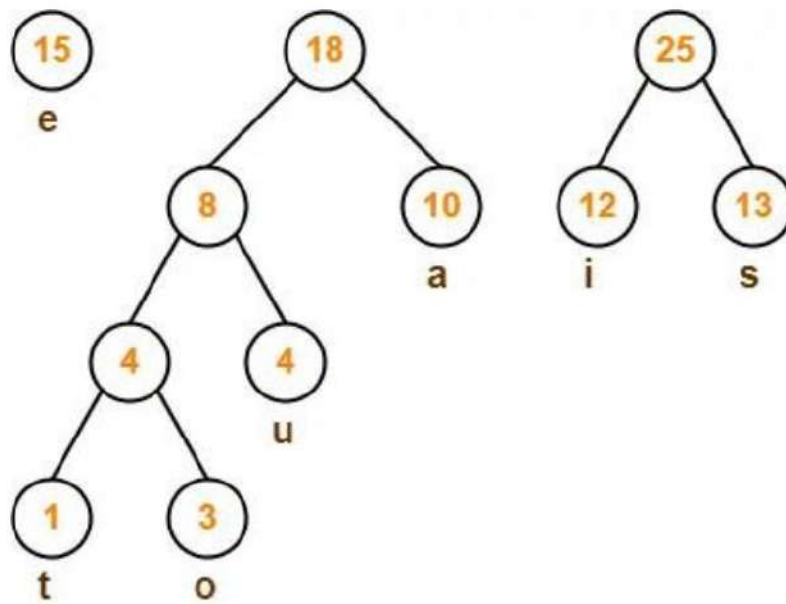
Step-04:



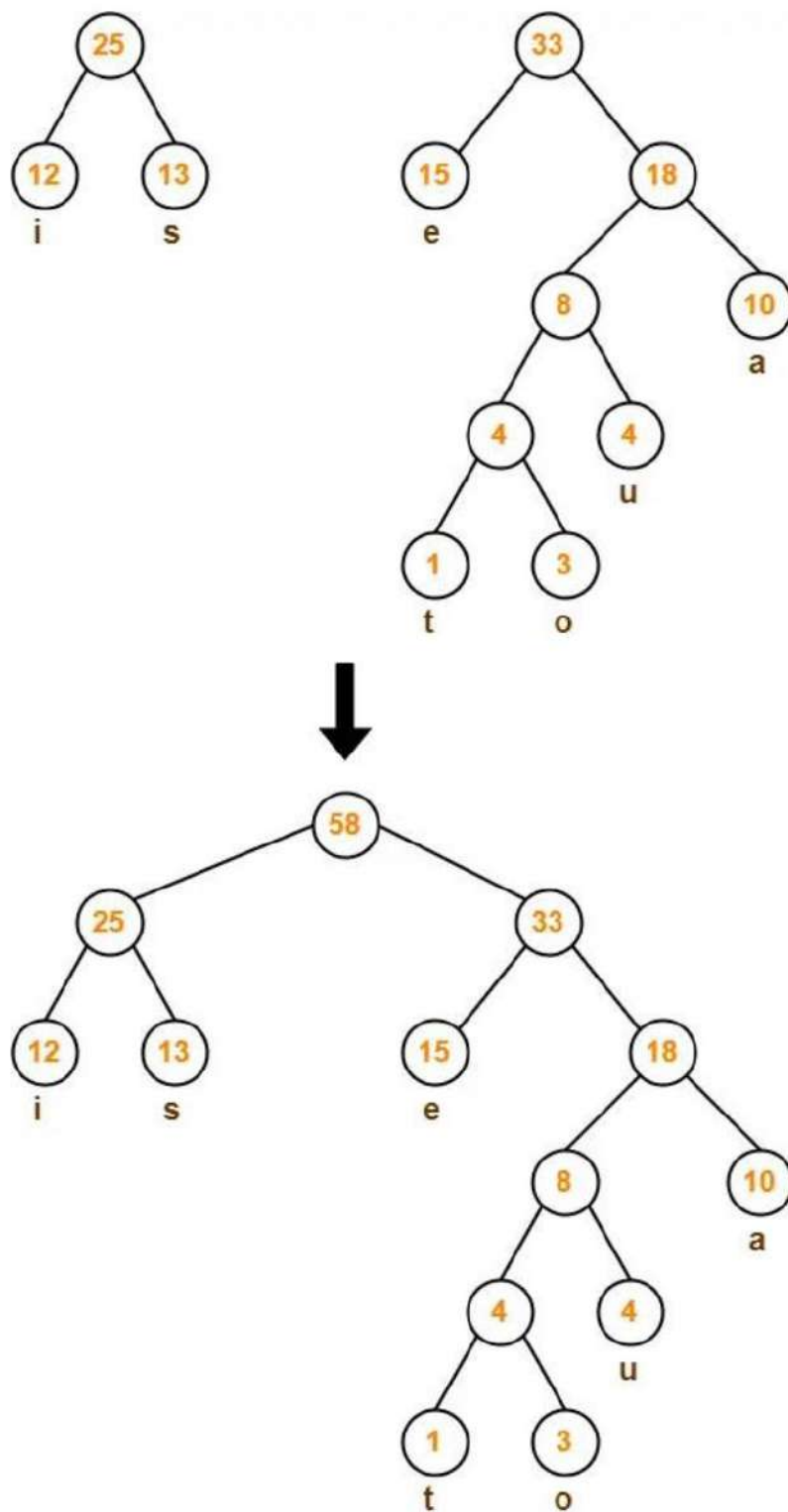
Step-05:



Step-06:



Step-07:

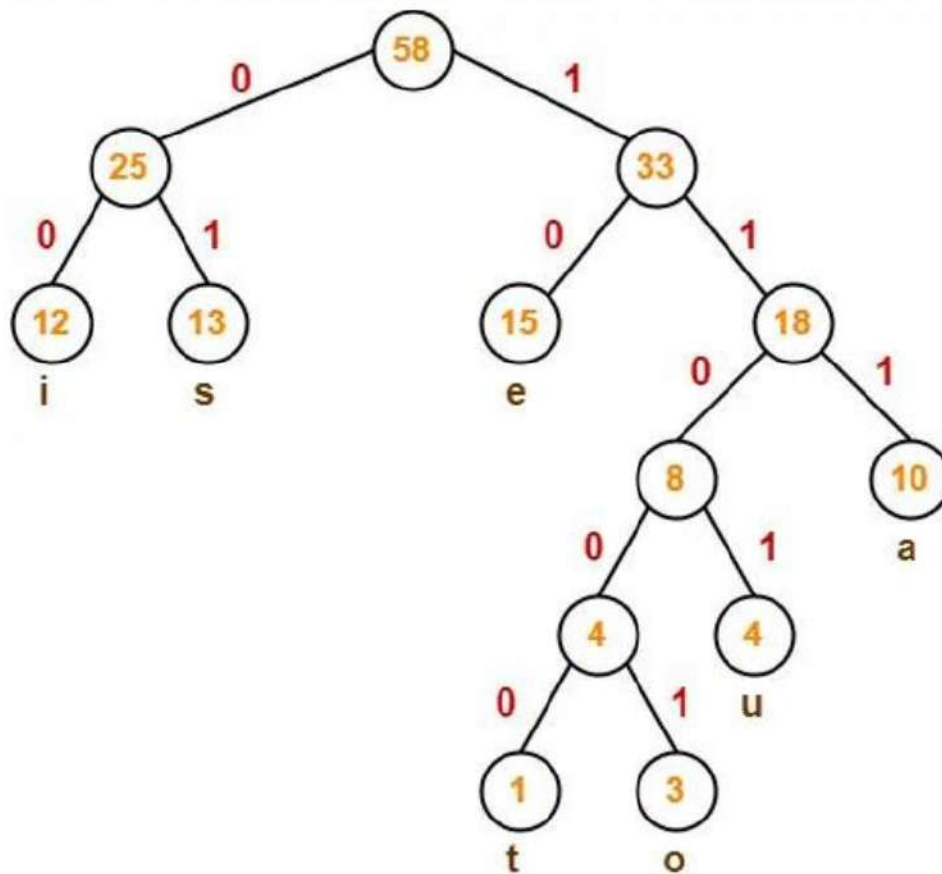


Huffman Tree

Now,

- We assign weight to all the **edges** of the constructed Huffman Tree.
- Let us assign **weight '0'** to the left edges and **weight '1'** to the right edges.

After assigning weight to all the edges, the modified Huffman Tree is-



Huffman Tree

Huffman Code For Characters-

⇒ To write Huffman Code for any character, traverse the Huffman Tree from root node to the leaf node of that character.

Following this rule, the Huffman Code for each character is-

- a = 111
- e = 10
- i = 00
- o = 11001
- u = 1101
- s = 01
- t = 11000

From here, we can observe-

- Characters occurring less frequently in the text are assigned the larger code.
- Characters occurring more frequently in the text are assigned the smaller code.

Master Theorem

Master Theorem

⇒ Master's Theorem is a popular method for solving the recurrence relations.

$$T(n) = a T\left(\frac{n}{b}\right) + \theta(n^k \log^p n)$$

Here, $a \geq 1$, $b > 1$, $k \geq 0$ and p is a real number.

Master Theorem Cases-

⇒ To solve recurrence relations using Master's theorem, we compare a with b^k . Then, we follow the following cases-

Case-01:

If $a > b^k$, then $T(n) = \theta(n^{\log_b a})$

Case-02:

If $a = b^k$ and

- If $p < -1$, then $T(n) = \theta(n^{\log_b a})$
- If $p = -1$, then $T(n) = \theta(n^{\log_b a} \cdot \log^2 n)$
- If $p > -1$, then $T(n) = \theta(n^{\log_b a} \cdot \log^{p+1} n)$

Case-03:

If $a < b^k$ and

- If $p < 0$, then $T(n) = O(n^k)$
- If $p \geq 0$, then $T(n) = \theta(n^k \log^p n)$

PRACTICE PROBLEMS BASED ON MASTER THEOREM

Problem-01:

⇒ Solve the following recurrence relation using Master's theorem- $T(n) = 3T(n/2) + n^2$

Solution-

⇒ We compare the given recurrence relation with $T(n) = aT(\frac{n}{b}) + \theta(n^k \log^p n)$.

Then we have-

$$a = 3$$

$$b = 2$$

$$k = 2$$

$$p = 0$$

Now, $a = 3$ and $b^k = 2^2 = 4$.

Clearly, $a < b^k$.

So, we follow case-03.

Since $p = 0$, so we have- $T(n) = \theta(n^k \log^p n)$

$$T(n) = \theta(n^2 \log^0 n)$$

$$T(n) = \theta(n^2)$$

Problem-02:

⇒ Solve the following recurrence relation using Master's theorem- $T(n) = 2T(n/2) + n \log n$

Solution-

⇒ We compare the given recurrence relation with $T(n) = aT(\frac{n}{b}) + \theta(n^k \log^p n)$.

Then we have-

$$a = 2$$

$$b = 2$$

$$k = 1$$

$$p = 1$$

Now, $a = 2$ and $b^k = 2^1 = 2$.

Clearly, $a = b^k$.

So, we follow case-02.

Since $p > -1$, so we have- $T(n) = \theta (n^{\log_b a} \cdot \log^{p+1} n)$

$$T(n) = \theta (n^{\log_2 2} \cdot \log^{1+1} n)$$

$$T(n) = \theta (n \log^2 n)$$

Problem-03:

⇒ Solve the following recurrence relation using Master's theorem- $T(n) = 2T(n/4) + n^{0.51}$

Solution-

⇒ We compare the given recurrence relation with $T(n) = aT(\frac{n}{b}) + \theta (n^k \log^p n)$.

Then we have-

$$a = 2$$

$$b = 4$$

$$k = 0.51$$

$$p = 0$$

Now, $a = 2$ and $b^k = 4^{0.51} = 2.027$

Clearly, $a < b^k$.

So, we follow case-03.

Since $p > 0$, so we have- $T(n) = \theta (n^k \cdot \log^p n)$

$$T(n) = \theta (n^{0.51} \cdot \log^0 n)$$

$$T(n) = \theta (n^{0.51})$$

Problem-04:

⇒ Solve the following recurrence relation using Master's theorem- $T(n) = \sqrt{2}T(n/2) + \log n$

Solution-

⇒ We compare the given recurrence relation with $T(n) = aT(\frac{n}{b}) + \theta (n^k \log^p n)$.

Then we have-

$$a = \sqrt{2}$$

$$b = 2$$

$$k = 0$$

$$p = 1$$

Now, $a = 2$ and $b^k = 2^0 = 1$

Clearly, $a > b^k$.

So, we follow **case-01**.

$$\text{so we have- } T(n) = \theta(n^{\log_b a})$$

$$T(n) = \theta(n^{\log_2 \sqrt{2}})$$

$$T(n) = \theta(n^{0.5})$$

$$T(n) = \theta(\sqrt{n})$$

Problem-05:

⇒ Solve the following recurrence relation using Master's theorem- $T(n) = 8T(n/4) + n^2 \log n$

Solution-

⇒ We compare the given recurrence relation with $T(n) = aT(\frac{n}{b}) + \theta(n^k \log^p n)$.

Then we have-

$$a = 8$$

$$b = 4$$

$$k = 2$$

$$p = 1$$

Now, $a = 8$ and $b^k = 4^2 = 16$

Clearly, $a < b^k$.

So, we follow **case-03**.

Since $p > 0$, so we have- $T(n) = \theta(n^k \log^p n)$

$$T(n) = \theta(n^2 \log^1 n)$$

$$T(n) = \theta(n^2 \log n)$$

Problem-5.2:

⇒ Solve the following recurrence relation using Master's theorem- $T(n) = 8T(n/4) - n^2 \log n$

Solution-

⇒ We compare the given recurrence relation with $T(n) = aT(\frac{n}{b}) + \theta(n^k \log^p n)$.

Then we have-

- The given recurrence relation does not correspond to the general form of Master's theorem.
- So, it cannot be solved using Master's theorem.

Problem-06:

⇒ Solve the following recurrence relation using Master's theorem- $T(n) = 3T(n/3) + n/2$

Solution-

⇒ We write the given recurrence relation as $T(n) = 3T(n/3) + n$.

⇒ This is because in the general form, we have θ for function $f(n)$ which hides constants in it.

Now, we can easily apply Master's theorem.

⇒ We compare the given recurrence relation with $T(n) = aT(\frac{n}{b}) + \theta(n^k \log^p n)$.

Then we have-

$$a = 3$$

$$b = 3$$

$$k = 1$$

$$p = 0$$

Now, $a = 3$ and $b^k = 3^1 = 3$.

Clearly, $a = b^k$.

So, we follow **case-02**.

Since $p > -1$, so we have- $T(n) = \theta(n^{\log_b a} \cdot \log^{p+1} n)$

$$T(n) = \theta(n^{\log_3 3} \cdot \log^{0+1} n)$$

$$T(n) = \theta(n \log n)$$

Recursion Tree

Recursion Tree

- Like **Master's Theorem**, Recursion Tree is another method for solving the recurrence relations.
- A recursion tree is a tree where each node represents the cost of a certain recursive sub-problem.
- We sum up the values in each node to get the cost of the entire algorithm.

Steps to Solve Recurrence Relations Using Recursion Tree Method-

Step-01:

⇒ Draw a **recursion tree** based on the given **recurrence relation**.

Step-02:

⇒ Determine-

- **Cost** of each level
- Total number of **levels** in the recursion tree
- Number of **nodes in the last level**
- **Cost of the last level**

Step-03:

⇒ **Add cost of all the levels** of the recursion tree and simplify the expression so obtained in terms of asymptotic notation.

PRACTICE PROBLEMS BASED ON RECURSION TREE-

Problem-2:

⇒ Solve the following recurrence relation using recursion tree method- $T(n) = 2T(n/2) + n$

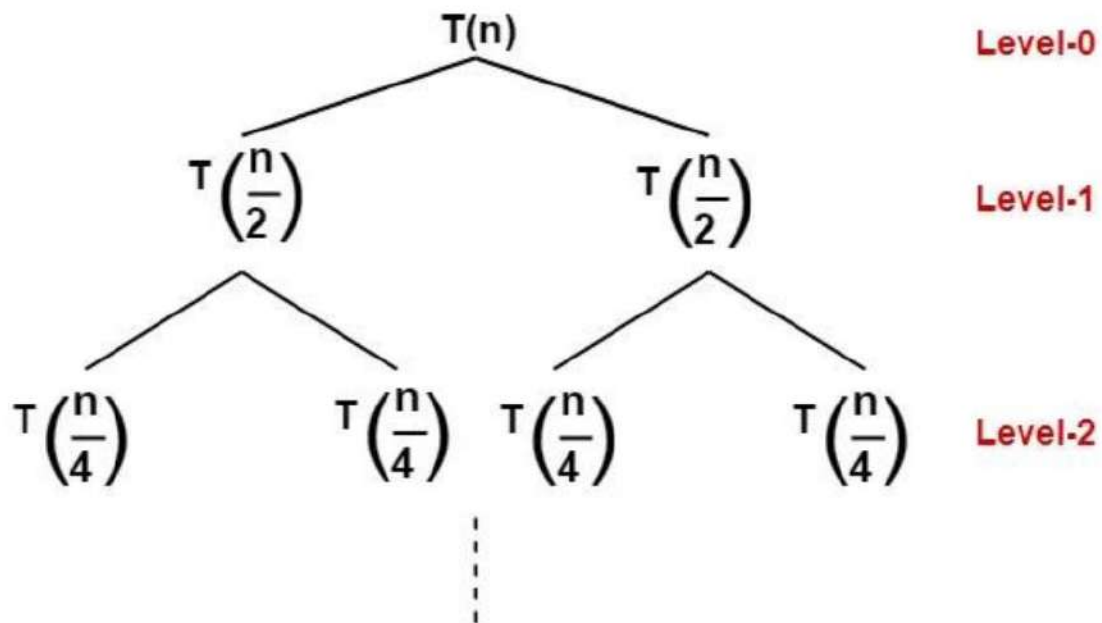
Solution-

Step-01:

⇒ Draw a recursion tree based on the given recurrence relation.

The given recurrence relation shows-

- A problem of size n will get divided into 2 sub-problems of size $n/2$.
- Then, each sub-problem of size $n/2$ will get divided into 2 sub-problems of size $n/4$ and so on.
- At the bottom most layer, the size of sub-problems will reduce to 1.



Step-02:

⇒ Determine **cost of each level**-

- Cost of level-0 = n
- Cost of level-1 = $n/2 + n/2 = n$
- Cost of level-2 = $n/4 + n/4 + n/4 + n/4 = n$ and so on.

Step-03:

⇒ Determine **total number of levels** in the recursion tree-

- Size of sub-problem at level-0 = $n/2^0$
- Size of sub-problem at level-1 = $n/2^1$
- Size of sub-problem at level-2 = $n/2^2$

Continuing in similar manner, we have-

Size of sub-problem at level- i = $n/2^i$

Suppose at **level- x** (last level), size of sub-problem becomes 1. Then-

$$\frac{n}{2^x} = 1$$

$$2^x = n$$

Taking **log** on both sides, we get- **$x \log 2 = \log n$**

$$x = \log_2 n$$

∴ **Total number of levels in the recursion tree = $\log_2 n + 1$**

Step-04:

⇒ Determine number of nodes in the last level-

- Level-0 has 2^0 nodes i.e. 1 node
- Level-1 has 2^1 nodes i.e. 2 nodes
- Level-2 has 2^2 nodes i.e. 4 nodes

Continuing in similar manner, we have-Level- $\log_2 n$ has $2^{\log_2 n}$ nodes i.e. n nodes

Step-05:

⇒ Determine cost of last level-

$$\text{Cost of last level} = n \times T(1) = \theta(n)$$

Step-06:

⇒ Add costs of all the levels of the recursion tree and simplify the expression so obtained in terms of asymptotic notation-

$$T(n) = \underbrace{\{ n + n + n + \dots \}}_{\text{For } \log_2 n \text{ levels}} + \theta(n)$$

For $\log_2 n$ levels

$$= n \times \log_2 n + \theta(n)$$

$$= n \log_2 n + \theta(n)$$

$$= \theta(n \log n)$$

⇒ **Summary:**

$n=8$	$n=2^3$	3 levels	we need 3.cn
$n=16$	$n=2^4$	4 levels	we need 4.cn
$n=32$	$n=2^5$	5 levels	we need 5.cn
$n=64$	$n=2^6$	6 levels	we need 6.cn
n	$n=2^k$	$\log_2 n$ levels	we need $\log_2 n . cn$

Time complexity:

$$= O(c * n \log_2 n)$$

$$= O(n * \log n)$$

Problem-2:

⇒ Solve the following recurrence relation using recursion tree method- $T(n) = T(n/5) + T(4n/5) + n$

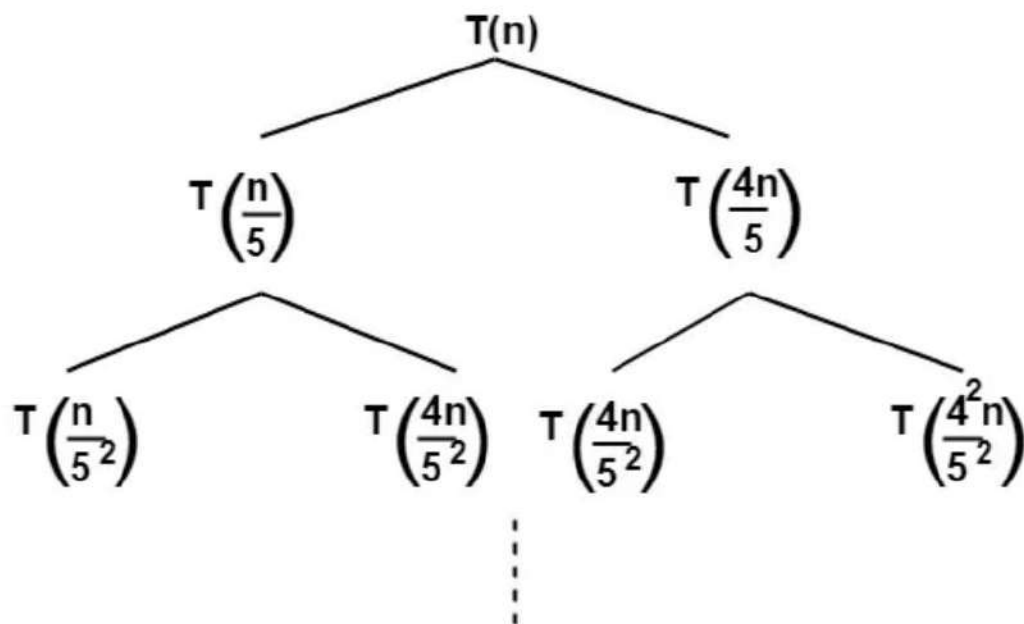
Solution-

Step-01:

⇒ Draw a recursion tree based on the given recurrence relation.

The given recurrence relation shows-

- A problem of size n will get divided into 2 sub-problems- one of size $n/5$ and another of size $4n/5$.
- Then, sub-problem of size $n/5$ will get divided into 2 sub-problems- one of size $n/5^2$ and another of size $4n/5^2$.
- On the other side, sub-problem of size $4n/5$ will get divided into 2 sub-problems- one of size $4n/5^2$ and another of size $4^2n/5^2$ and so on.
- At the bottom most layer, the size of sub-problems will reduce to 1.



Step-02:

⇒ Determine cost of each level-

- Cost of level-0 = n
- Cost of level-1 = $n/5 + 4n/5 = n$
- Cost of level-2 = $n/5^2 + 4n/5^2 + 4n/5^2 + 4^2n/5^2 = n$

Step-03:

⇒ Determine **total number of levels** in the recursion tree-

- Size of sub-problem at level-0 = $(4/5)^0 n$
- Size of sub-problem at level-1 = $(4/5)^1 n$
- Size of sub-problem at level-2 = $(4/5)^2 n$

Continuing in similar manner, we have-

Size of sub-problem at level-i = $(4/5)^i n$

Suppose at **level-x** (last level), size of sub-problem becomes 1. Then-

$$(4/5)^x n = 1$$

$$(4/5)^x = 1/n$$

Taking **log** on both sides, we get- $x \log(4/5) = \log(1/n)$

$$x = \log_{5/4} n$$

∴ **Total number of levels in the recursion tree = $\log_{5/4} n + 1$**

Step-04:

⇒ Determine number of nodes in the last level-

- Level-0 has 2^0 nodes i.e. 1 node
- Level-1 has 2^1 nodes i.e. 2 nodes
- Level-2 has 2^2 nodes i.e. 4 nodes

Continuing in similar manner, we have- **Level- $\log_{5/4} n$ has $2^{\log_{5/4} n}$ nodes i.e. n nodes**

Step-05:

⇒ Determine cost of last level-

$$\text{Cost of last level} = 2^{\log_{5/4} n} \times T(1) = \theta(2^{\log_{5/4} n}) = \theta(n^{\log_{5/4} 2})$$

Step-06:

⇒ Add costs of all the levels of the recursion tree and simplify the expression so obtained in terms of asymptotic notation-

$$T(n) = \{ \underbrace{n + n + n + \dots}_{\text{For } \log_{5/4} n \text{ levels}} \} + \theta(n^{\log_{5/4} 2})$$

For $\log_{5/4} n$ levels

$$= n \log_{5/4} n + \theta(n^{\log_{5/4} 2})$$

$$= \theta(n \log_{5/4} n)$$

Algorithm Analysis

Time Complexity Analysis:

Example:

```
A() {  
    int i = 0;  
    for (i = 1 to n) {  
        PB("Hi")  
    }  
}
```

→ n times

∴ the complexity is $O(n)$

Example:

```
A() {  
    int i, j;  
    for (i = 1 to n) {  
        for (j = 1 to n) {  
            PB("Neymer")  
        }  
    }  
}
```

→ n times
→ n times
→ $n \times n = n^2$ times

∴ Analyzing ∴ the complexity is $O(n)$

Example:-

```

A() {
    i = 1, s = 1;
    while (s <= n) {
        i++;
        s = s + i;
        printf("messi");
    }
}

```

Analyzing:-

$i = 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad \dots \quad K$
 $s = 1 \quad 3 \quad 6 \quad 10 \quad 15 \quad 21 \quad \dots \quad n = \frac{K(K+1)}{2}$

→ Sum of K natural numbers.

Now To stop the loop:-

$$\Rightarrow \frac{K(K+1)}{2} > n$$

$$\Rightarrow \frac{K^2 + K}{2} \geq n$$

∴ The time complexity $K = O(\sqrt{n})$

Example:-

```

A() {
    int i = 1;
    for (i = 1; i <= n; i++) {
        printf("Rolando");
    }
}

```

→ $i = \sqrt{n}$

→ $\sqrt{n}$ times

∴ The Complexity $O(\sqrt{n})$

Example:-

A() {

for(int i=1; i<=n; i++) {

for(int j=1; j<=i; j++) {

for(int k=1; k<=100; k++) {

printf("mbappe");

}

Analyzing:-

if $i=1$ $j=1$ times $k=1 \times 100$ times
 $i=2$ $j=2$ times $k=2 \times 100$ times
 $i=3$ $j=3$ times $k=3 \times 100$ times
 $\therefore i=n$
 $j=n$ times
 $k=n \times 100$ times, ...

$\therefore$ total No. of times printf function execute.

$$= 1 \times 100 \text{ times} + 2 \times 100 + 3 \times 100 + \dots + n \times 100$$

$$= 100(1+2+3+4+\dots)$$

$$= 100 \times \frac{n(n+1)}{2} \rightarrow \text{Sum of } n \text{ natural Number.}$$

$$= 100 \times \frac{n^2+n}{2}$$

$\therefore$ The time complexity is $O(n^2)$

Example:-
C++

for (i = 1; i <= n; i++)

{
A(i)
for (j = 1; j <= i^2; j++)
for (k = 1; k <= n/2; k++)
print("Real Madrid");
}

if Analyzing:-
C++

if i = 1
j = 1 time
k = 1 x $\frac{n}{2}$ times

i = 2
j = 2^2 times

k = 2^2 x $\frac{n}{2}$ times

i = 3
j = 3^2 times

k = 3^2 x $\frac{n}{2}$ times

$\therefore i = n$

j = n^2

k = n^2 x $\frac{n}{2}$ times

$\therefore$ The total No. of print function execute

= $1 \times \frac{n}{2}$ times + $2^2 \times \frac{n}{2}$ times + $3^2 \times \frac{n}{2}$ times
+ + $n^2 \times \frac{n}{2}$ times

= $\frac{n}{2} (1^2 + 2^2 + 3^2 + 4^2 + \dots + n^2)$

= $\frac{n}{2} \left[\frac{n(n+1)(2n+1)}{6} \right] \rightarrow$ Sum of Squares of n natural numbers.

$$\begin{aligned}
 &= \frac{n^2(n+1)(2n+1)}{2 \times 6} \\
 &= \frac{(n^3+n^2)(2n+1)}{12} \\
 &= \frac{2n^4+n^3+2n^3+n^2}{12} \\
 &= \frac{2n^4+3n^3+n^2}{12}
 \end{aligned}$$

$\therefore$ The time complexity $= O(n^4)$

Example:-

A() {

for(i=1; i<n; i=i*2) {

print("Hala Madrid");

}

Analyzing:-

$$\begin{aligned}
 i &= 1, 2, 4, 8, \dots, n \\
 &= 2^0, 2^1, 2^2, 2^3, \dots, 2^k
 \end{aligned}$$

$\therefore$ The total number of print function execute

$$\Rightarrow 2^k = n$$

$$\Rightarrow \log_2 2^k = \log_2 n$$

$$\Rightarrow k = \log_2(n)$$

$\therefore$ The time complexity is $O(\log n)$

Example:-
non

A() {

for($i = \frac{n}{2}$; $i \leq n$; $i++$) $\rightarrow \frac{n}{2}$ times

for($j = 1$; $j \leq \frac{n}{2}$; $j++$) $\rightarrow \frac{n}{2}$ times

for($k = 1$; $k \leq n$; $k = k * 2$) $\rightarrow \log_2 n$ times

print("Visca Barca");

} } }

Analyzing:-
non

$\therefore$ the total numbers of print function execute

$$\begin{aligned} &= \frac{n}{2} \times \frac{n}{2} \times \log_2 n \\ &= \frac{n^2}{2} \log_2 n \end{aligned}$$

$\therefore$ The time complexity = $O(n^2 \log_2 n)$

Example:-
non

A() {

for($i = \frac{n}{2}$; $i \leq n$; $i++$) $\rightarrow \frac{n}{2}$ times

for($j = 1$; $j \leq n$; $j = j * 2$) $\rightarrow \log_2 n$ times

for($k = 1$; $k \leq n$; $k = k * 2$) $\rightarrow \log_2 n$ times

print("SIUUU....");

} } }

Ex Analyzing:-

$\therefore$ No. of print function will be executed.

$$= \frac{n}{2} \times \log_2 n \times \log_2 n$$

$$= \frac{n}{2} \times (\log_2 n)^2$$

$\therefore$ the time complexity = $O(n \cdot \log^2 n)$

Ex Example:-

A() {

while (n > 1) {

$$n = \frac{n}{2};$$

}

Ex Analyzing:-

if:- $n = 2^1$

$$\therefore \frac{n}{2} = \frac{2}{2} = 1$$

$$\left. \begin{array}{l} n = 2^2 \\ \frac{n}{2} = \frac{4}{2} = 2 \\ \quad = \frac{2}{2} = 1 \end{array} \right\}$$

$$\left. \begin{array}{l} n = 2^3 \\ \frac{n}{2} = \frac{8}{2} = 4 \\ \quad = \frac{4}{2} = 2 \\ \quad \quad = \frac{2}{2} = 1 \end{array} \right\}$$

$\therefore n = 2^k$

$$\Rightarrow \log_2 2^k = \log_2 n$$

$$\therefore k = \log_2 n$$

$\therefore$ the time complexity = $O(\log n)$

Example:—

$A() \{$

$\text{for}(i=1; i \leq n; i++) \rightarrow n \text{ times}$

$\text{for}(j=1; j \leq n; j=i+j) \{$

$\text{print}("Vini");$

$\} \}$

✶ Analyzing:—

if:- $i=1;$
 $j=1 \text{ to } n$

$\therefore j = n \text{ times}$

$j=1+i=1+1=2, 3, 4, \dots$

$i=2$

$\therefore j=1 \text{ to } n$

$\therefore j = \frac{n}{2} \text{ times}$

$j=1+2=3, 5, 7, \dots$

$i=3$

$j=1 \text{ to } n$

$\therefore j = \frac{n}{3}$

Since, $j=j+i=1+3=4, 8, 11, \dots$

$\therefore i=k$

$j=1 \text{ to } n$

$\therefore j = \frac{n}{k} \text{ times}$

Since, $j=j+i$

$i=n$

$j=1 \text{ to } n$

$\therefore j = \frac{n}{n} = 1 \text{ times}$

$\therefore$ total no. of print function will execute is

$$= (n + \frac{n}{2} + \frac{n}{3} + \dots + \frac{n}{k} + \dots + \frac{n}{n})$$

$$= n(1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n})$$

$$= n(\sum_{k=1}^n \frac{1}{k})$$

$$= n(\log n)$$

$\therefore$ Time Complexity $O(n \log n)$

$$\sum_{k=1}^n \frac{1}{k} = n$$

Example:-

```

A() {
    int n = 2k;
    for (i = 1; i <= n; i++) } → n times
    int j = 2;
    while (j <= n) {
        j = j * 2;
        print("Man City");
    }
}

```

Analyzing:-

<u>IF</u> $k = 1$ $n = 2^1 = 2$ $j = 2, 4$ $\text{print}() \rightarrow 2 \text{ times} \times n$	$k = 2$ $n = 4$ $j = 2, 4, 16$ $\text{print}() \rightarrow n \times 3 \text{ times}$	$k = 3$ $n = 8$ $j = 2, 4, 16, 256$ $\text{print}() \rightarrow n \times 4 \text{ times}$
--	--	---

∴ $k = k$
 $n = 2^k$ ————— (i)
 $j = 2^1, 2^2, 2^4, 2^{16} \dots$

$\text{print}() \rightarrow n \times (k+1) \text{ times}$. ————— (ii)

∴ From eq (i) & (ii)

$$\begin{aligned}
 2^k &= n \\
 \log_2 2^k &= \log_2 n \\
 k &= \log_2 n \\
 \log_2 2^k &= \log_2 \log_2 n \\
 \therefore k &= \log_2 \log_2 n
 \end{aligned}$$

Now, put the value of k in eq (1) $\Rightarrow$

$\therefore$ total print function will be executed:-

$$= n (\log_2 \log_2 n + 1) \text{ times.}$$

$\therefore$ The time complexity is

$$= O(n \log \log n)$$

Ans:-

Algorithm Analysis:-

Example: 1:-

```
Sum(a, b) {  
    return a + b; } → (1) time.
```

Example: 2:-

```
evenOdd(n) {  
    if (n % 2 == 0) {  
        print("Even"); } → (1) time  
    else {  
        print("Odd"); } → (1) time  
}
```

Example: 3:-

```
ArrayFirstElement(a, n) {  
    Element = a[0];  
    return Element; } → (2) times  
}
```

Example: 4:-

```
print(n) {  
    for i = 1 to n {  
        print i; } → (n) times  
    }
```


Example 5:-

```
print()
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        print("Hi");
    }
}
```

→ (n) times
→ (n) times } (n²) times

Example 6:-

```
print()
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        for (int k = 1; k <= n; k++) {
            print("Hello");
        }
    }
}
```

→ (n) times
→ (n) times } (n³) times
→ (n) times

Example 7:-

```
print()
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        for (int k = 1; k <= 100; k++) {
            print("Hi");
        }
    }
}
```

→ (n) times
→ (n) times
→ $\left[\frac{n(n+1)}{2} \right]$ times } (n²) times

Analysing Ex: 7:-

For: $i=1$
 $j=1$
 $k \leq 100$

For: $i=2$
 $j=1, 2$
 $k_2 = 100 + 100 = 200$
 $= 2 \times 100$

For: $i=3$
 $j=1, 2, 3$
 $k_3 = 100 + 100 + 100$
 $= 300 = 3 \times 100$

That means:-

$i=n$
 $j=1, 2, \dots, n$
 $k_n = n \times 100$

∴ Total Number of print function performed

$$\Rightarrow k_1 + k_2 + k_3 + \dots + k_n$$

$$= 100 + 2 \times 100 + 3 \times 100 + \dots + n \times 100$$

$$= 100 (1 + 2 + 3 + \dots + n)$$

$$= 100 \times \left[\frac{n(n+1)}{2} \right] = n^2$$

Example 8:-

```

print()
for (i=1; i<=n; i++) → (n) times
  for (j=1; j<=i; j++) → (n^2) times
    for (k=1; k<=n/2; k++) → (n^4) times
      print("Q7");
  }
}

```


Analyzing Ex: 8:-

for $i = 1$
 $j = 1^2 = 1$

$$k_1 = 1 \times \frac{n}{2}$$

for $i = 2$
 $j = 2^2 = 4$

$$k_2 = 4 \times \frac{n}{2}$$

for $i = 3$
 $j = 3^2 = 9$

$$k_3 = 9 \times \frac{n}{2}$$

So, Now for $i = n$
 $j = n^2$

$$k_n = n^2 \times \frac{n}{2}$$

$\therefore$ Total Number of print function performed

$$\Rightarrow k_1 + k_2 + k_3 + \dots + k_n$$

$$\Rightarrow 1 \times \frac{n}{2} + 4 \times \frac{n}{2} + 9 \times \frac{n}{2} + \dots + n^2 \times \frac{n}{2}$$

$$\Rightarrow \frac{n}{2} (1 + 4 + 9 + \dots + n^2)$$

$$\Rightarrow \frac{n}{2} (1 + 2^2 + 3^2 + 4^2 + \dots + n^2)$$

$$\Rightarrow \frac{n}{2} \left(\frac{n(n+1)(2n+1)}{6} \right)$$

Example: 9:-

```
print()
```

```
for (int i = 1; i <= n; i = i * 2)
```

```
    print("messi");
```

```
}
```

```
}
```

$\rightarrow (\log n)$ times.

Ex Analyzing Ex: 1:-

For $i = 1, 2, 4, 8, 16, \dots, n$

So, pattern = $2^0, 2^1, 2^2, 2^3, 2^4, \dots, 2^m$

So, $2^m = n$

$\therefore m = \log_2 n$

Time Complexity for
Recursive problem:-

Example:-
mon

$A(n)$ {

if ($\swarrow$) {

prin + $(A(\frac{n}{2}) + A(\frac{n}{2}))$;

}

$\therefore T(n) = c + 2T(\frac{n}{2})$

☒ Recurrence Relation
of Recursive Algorithms:-

Example: 1:-

$A(n)$

$\text{if } (n > 1)$

$\text{return } (A(n-1));$

$\}$

$$\therefore T(n) = c + T(n-1)$$

We can solve the Recursive function in two ways

1. Back Substitution method.

2. Master Method.

Back Substitution:-

In given:-

$$T(n) = c + T(n-1) \quad \text{--- (i)}$$

$$\begin{aligned} \therefore T(n-1) &= c + [c + T(n-2)] \\ &= 2c + T(n-2) \quad \text{--- (ii)} \end{aligned}$$

$$\begin{aligned} \therefore T(n-2) &= 2c + [c + T(n-3)] \\ &= 3c + T(n-3) \quad \text{--- (iii)} \end{aligned}$$

$\therefore$ Put the value of eq (ii) & (iii) in eq (i)

$$\begin{aligned} T(n) &= c + T(n-1) \\ &= 2c + T(n-2) \\ &= 3c + T(n-3) \\ &= kc + T(n-k) \quad \text{--- (iv)} \end{aligned}$$

The loop are stop when $n=1$

$\therefore$ Let,

$$n-k=1$$

$$\therefore k = n-1$$

$\therefore$ Now, put the value of k in eq (iv):—

$$T(n) = (n-1)c + T(1)$$

$$= nc - c + 1$$

$$= nc$$

$\therefore$ Time Complexity of $T(n) = O(n)$

Example:- 2:-

$$T(n) = \begin{cases} n + T(n-1) & ; n > 1 \\ 1 & ; n = 1 \end{cases}$$

Find the time complexity using back-substitution method.

$\Rightarrow$ In given:-

$$T(n) = n + T(n-1) \quad \text{--- (i)}$$

$$\begin{aligned} \therefore T(n-1) &= n-1 + [T(n-2)] \\ &= (n-1) + T(n-2) \end{aligned} \quad \text{--- (ii)}$$

$$\begin{aligned} T(n-2) &= (n-2) + [T(n-3)] \\ &= (n-2) + T(n-3) \end{aligned} \quad \text{--- (iii)}$$

$\therefore$ Substitute the value of (ii) & (iii) in eq (i)

$$\therefore T(n) = n + T(n-1)$$

$$= n + (n-1) + T(n-2)$$

$$= n + (n-1) + (n-2) + T(n-3)$$

$$= n + (n-1) + (n-2) + \dots + (n-k) + T(n-k+1) \quad \text{(iv)}$$

Let, $n-k+1 = 1$

$$\therefore k = n-2$$

$\therefore$ put the value of k in eq (iv):

$$= n + (n-1) + (n-2) + \dots + (n-n+2) + T(1)$$

$$= n + (n-1) + (n-2) + \dots + 2 + 1$$

$$= \boxed{\frac{n(n+1)}{2}}$$

$\rightarrow$ Sum of n natural numbers.

$$= \frac{n^2 + n}{2}$$

$\therefore$

$\therefore$ The complexity of $T(n) = O(n^2)$

Substitution Method

Example:-

Example 1:-

$$T(n) = n + T(n-1); \text{ if } (n > 1) \\ = 1; \text{ if } (n = 1)$$

$$\begin{aligned} \therefore T(n) &= n + (n-1) + T(n-2) \\ &= n + n-1 + T(n-2) \\ &= 2n-1 + T(n-2) \\ &= 2n-1 + T(n-3) + (n-2) \\ &= 2n + n-1-2 + T(n-3) \\ &= 3n-3 + T(n-3) \end{aligned}$$

$$= kn-k + T(n-k) \quad (k=3)$$

Let $n-k=1$

$$\therefore k=n-1$$

Substitute the values:-

$$\begin{aligned} T(n) &= (n-1)n - (n-1) + T(n-n+1) \\ &= n^2 - n - n + 1 + T(1) \\ &= n^2 + 1 + 1 \end{aligned}$$

$$\therefore T(n) = n^2 + 2$$

$\therefore$ The complexity of $T(n) = O(n^2)$

Exo 2:-

$$T(n) = \begin{cases} 1 + T(n-1) & ; \text{if } n > 1 \\ 1 & ; \text{if } n = 1 \end{cases}$$

is given:

$$T(n) = 1 + T(n-1) \quad \text{--- (i)}$$

$$T(n-1) = 1 + T(n-2) \quad \text{--- (ii)}$$

$$T(n-2) = 1 + T(n-3) \quad \text{--- (iii)}$$

Now Substitute the values for this 3 eq

$$\begin{aligned} T(n) &= 1 + 1 + 1 + T(n-3) \\ &= 3 + T(n-3) \end{aligned}$$

$$\therefore T(n-k) = k + T(n-k) \quad \text{--- (iv)}$$

Stop time =

$$n - k = 1$$

$$\therefore k = n - 1$$

Now Substitute the value of k in eq (iv)

$$\Rightarrow n - 1 + T(n - n + 1)$$

$$\Rightarrow n - 1 + T(1)$$

$$\Rightarrow n - 1 + 1$$

$$\Rightarrow n$$

Time Complexity $T(n) = O(n)$

Masters Method: (Short Cut)

Conditions:-

General Format:-

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Where,

$$a \geq 1, b \geq 1$$

and, $f(n)$ is positive

Cases:-

Case 1:- if, $f(n) < n^{\log_b a}$

Then, $T(n) = \Theta(n^{\log_b a})$

Case 2:- if, $f(n) = n^{\log_b a}$

Then, $T(n) = \Theta(n^{\log_b a} \cdot \log n)$

Case 3:-

if, $f(n) > n^{\log_b a}$

Then, $T(n) = \Theta(f(n))$

Example 1:-

$$\Rightarrow T(n) = 4T\left(\frac{n}{2}\right) + n$$

here,

$$a = 4$$

$$b = 2$$

$$f(n) = n$$

Now, check the condition:-

$$\Rightarrow n^{\log_b a}$$

$$\Rightarrow n^{\log_2 4}$$

$$\Rightarrow n^2$$

$$\therefore f(n) < n^2$$

$$\begin{aligned}\text{Then, } T(n) &= \Theta(n^{\log_b a}) \\ &= \Theta(n^{\log_2 4}) \\ &= \Theta(n^2)\end{aligned}$$

Ans:-

Example 2:-

$$\Rightarrow T(n) = 2T\left(\frac{n}{2}\right) + n$$

here,

$$a = 2$$

$$b = 2$$

$$f(n) = n$$

Now, check the condition:-

$$\Rightarrow n^{\log_b a}$$

$$\Rightarrow n^{\log_2 2}$$

$$\Rightarrow n^1$$

$$\Rightarrow n$$

$$\therefore f(n) = n^{\log_b a}$$

Then:-

$$\begin{aligned} T(n) &= \Theta(n^{\log_b a} \cdot \log n) \\ &= \Theta(n^{\log_2 2} \cdot \log n) \\ &= \Theta(n \log n) \end{aligned}$$

Ans:-

Examples 3:-

$$\Rightarrow T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

here,

$$a = 2,$$

$$b = 2$$

$$f(n) = n^2$$

Now, check the condition:-

$$\Rightarrow n^{\log_b a}$$

$$\Rightarrow n^{\log_2 2}$$

$$\Rightarrow n^1$$

$$\Rightarrow n$$

$$\therefore f(n) > n^{\log_b a}$$

Then

$$\begin{aligned} T(n) &= \Theta(f(n)) \\ &= \Theta(n^2) \end{aligned}$$

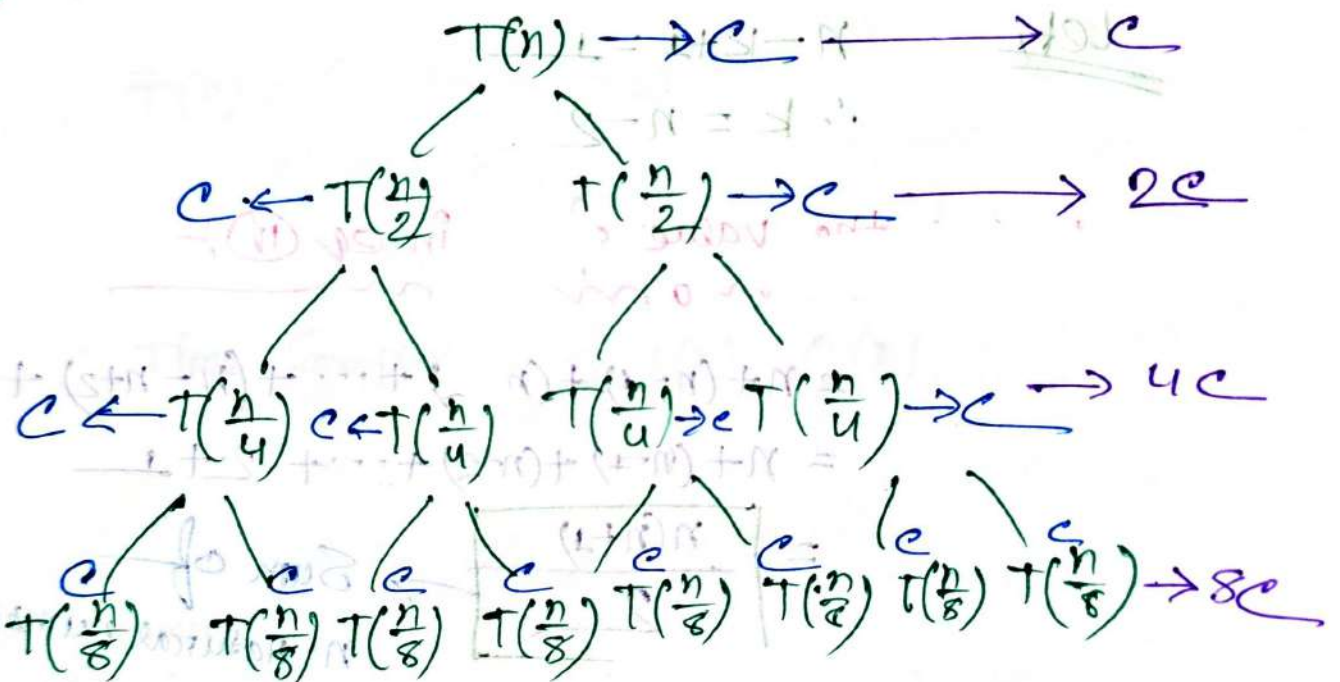
Ans:-

Recursion tree method

Example:-

$$T(n) = \begin{cases} 2T(\frac{n}{2}) + c; & n > 1 \\ c; & n = 1 \end{cases}$$

For each level
total c



Last level $T(\frac{n}{n}) \quad T(\frac{n}{n}) \quad T(\frac{n}{n}) \quad T(\frac{n}{n}) \dots \rightarrow nc$

$\therefore$ The total cost is =

$$\Rightarrow c + 2c + 4c + 8c + \dots + nc$$

$$\Rightarrow c(1 + 2 + 4 + 8 + \dots + n)$$

$$\Rightarrow c(2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^n)$$

$$\Rightarrow c(2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^k)$$

$$\Rightarrow c \left(\frac{1(2^{k+1} - 1)}{(2 - 1)} \right)$$

$$= c(2^{k+1} - 1)$$

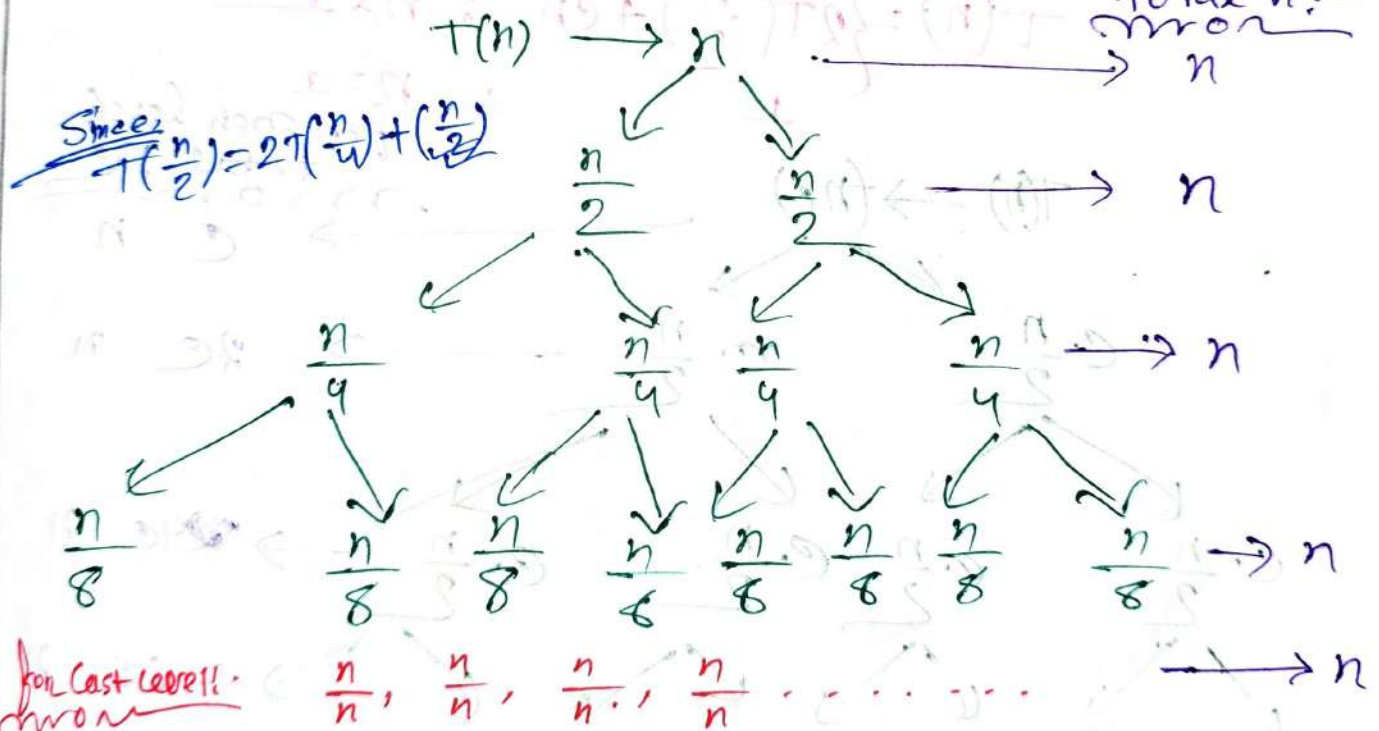
$$= c(2n - 1)$$

∴ The time complexity $T(n) = O(n)$

Example:-

$$T(n) = \begin{cases} 2T(\frac{n}{2}) + n & ; n > 1 \\ 1 & ; n = 1 \end{cases}$$

for each level
total n :-
mon



∴ total level:-

The series is:- $\frac{n}{2^0} + \frac{n}{2^1} + \frac{n}{2^2} + \frac{n}{2^3} + \dots + \frac{n}{2^k}$

$$\text{Total level} = k + 1 \quad \text{--- ①}$$

Let

$$\frac{n}{2^k} = 1$$

$$\therefore n = 2^k$$

$$\therefore k = \log_2 n$$

put it in eq ①:-

$$\text{Total level} = \log_2 n + 1$$

∴ The total cost for the recursive function is: mm o m o m o m o m

$$\Rightarrow \text{total ex total level} \\ \Rightarrow n \times (\log_2 n + 1)$$

∴ The time complexity $T(n) = O(n \cdot \log n)$

Example:-
mm o m

$$T(n) = \begin{cases} 2T\left(\frac{n}{2}\right) + cn & ; n > 1 \\ 1 & ; n = 1 \end{cases}$$

Do it yourself

Time Complexity
of Merge Sort

Merge Sort Time Complexity Using Substitution method

MergeSort(p, r)

{ if $p < r$

$q = \lfloor (p+r)/2 \rfloor$

MergeSort(p, q) $\rightarrow T(\frac{n}{2})$

MergeSort($q+1, r$) $\rightarrow T(\frac{n}{2})$

Merge(p, q, r) $\rightarrow n$

}

Merge(p, q, r)

{

$n_1 = q - p + 1 \rightarrow (1)$

$n_2 = r - q \rightarrow (1)$

Create Arrays $L[n_1+1]$ & $R[n_2+1]$

for $i = 0$ to n_1

do $L[i] = A[p+i] \rightarrow (\frac{n}{2})$

for $j = 0$ to n_2

do $R[j] = A[q+j+1] \rightarrow (\frac{n}{2})$

$L[n_1] = \text{infinity} \rightarrow (1)$

$R[n_2] = \text{infinity} \rightarrow (1)$

$i = 0, j = 0$

for $k = p$ to r

do if $L[i] \leq R[j]$

$A[k] = L[i]$

else

$A[k] = R[j]$

}

$$\therefore T(n) = \begin{cases} 2T\left(\frac{n}{2}\right) + n & ; \text{ if } n > 1 \\ 1 & ; \text{ if } n = 1 \end{cases}$$

$\therefore$ Substitute the values of ea.

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \\ &= 2\left\{2T\left(\frac{n}{4}\right) + \frac{n}{2}\right\} + n \\ &= 2 \times 2T\left(\frac{n}{4}\right) + n + n \\ &= 2 \times 2T\left(\frac{n}{4}\right) + 2n \\ &= 2 \times 2 \left\{2T\left(\frac{n}{8}\right) + \frac{n}{4}\right\} + 2n \\ &= 4 \left\{2T\left(\frac{n}{8}\right) + \frac{n}{4}\right\} + 2n \\ &= 8T\left(\frac{n}{8}\right) + n + 2n \\ &= 8T\left(\frac{n}{8}\right) + 3n \\ &= 2^3 T\left(\frac{n}{2^3}\right) + 3n \\ &= 2^k T\left(\frac{n}{2^k}\right) + kn \quad \text{--- (1)} \end{aligned}$$

let $\frac{n}{2^k} = 1$

$$\therefore 2^k = n$$

$$\therefore k = \log_2 n$$

Substitute the values in eq (1)

$$\begin{aligned} &= n T(1) + \log_2 n \times n \\ &= n + \log_2 n \times n \\ &= n + n \log_2 n \end{aligned}$$

$\therefore$ Time Complexity

$$O(n + n \log_2 n) = O(n \log_2 n + n) = O(n \log_2 n)$$

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \quad \text{--- (i)} \\ T\left(\frac{n}{2}\right) &= 2T\left(\frac{n}{4}\right) + \frac{n}{2} \quad \text{--- (ii)} \\ T\left(\frac{n}{4}\right) &= 2T\left(\frac{n}{8}\right) + \frac{n}{4} \quad \text{--- (iii)} \end{aligned}$$

Merge Sort Complexity
Using Master method:-

We have:-

$$T(n) = \begin{cases} 2T\left(\frac{n}{2}\right) + n & ; n > 1 \\ 1 & ; n = 1 \end{cases}$$

$$\therefore T(n) = 2T\left(\frac{n}{2}\right) + n$$

here,

$$a = 2$$

$$b = 2$$

$$f(n) = n$$

Now, Check the Condition:-

$$\Rightarrow n^{\log_b a}$$

$$\Rightarrow n^{\log_2 2}$$

$$\Rightarrow n^1$$

$$\Rightarrow n$$

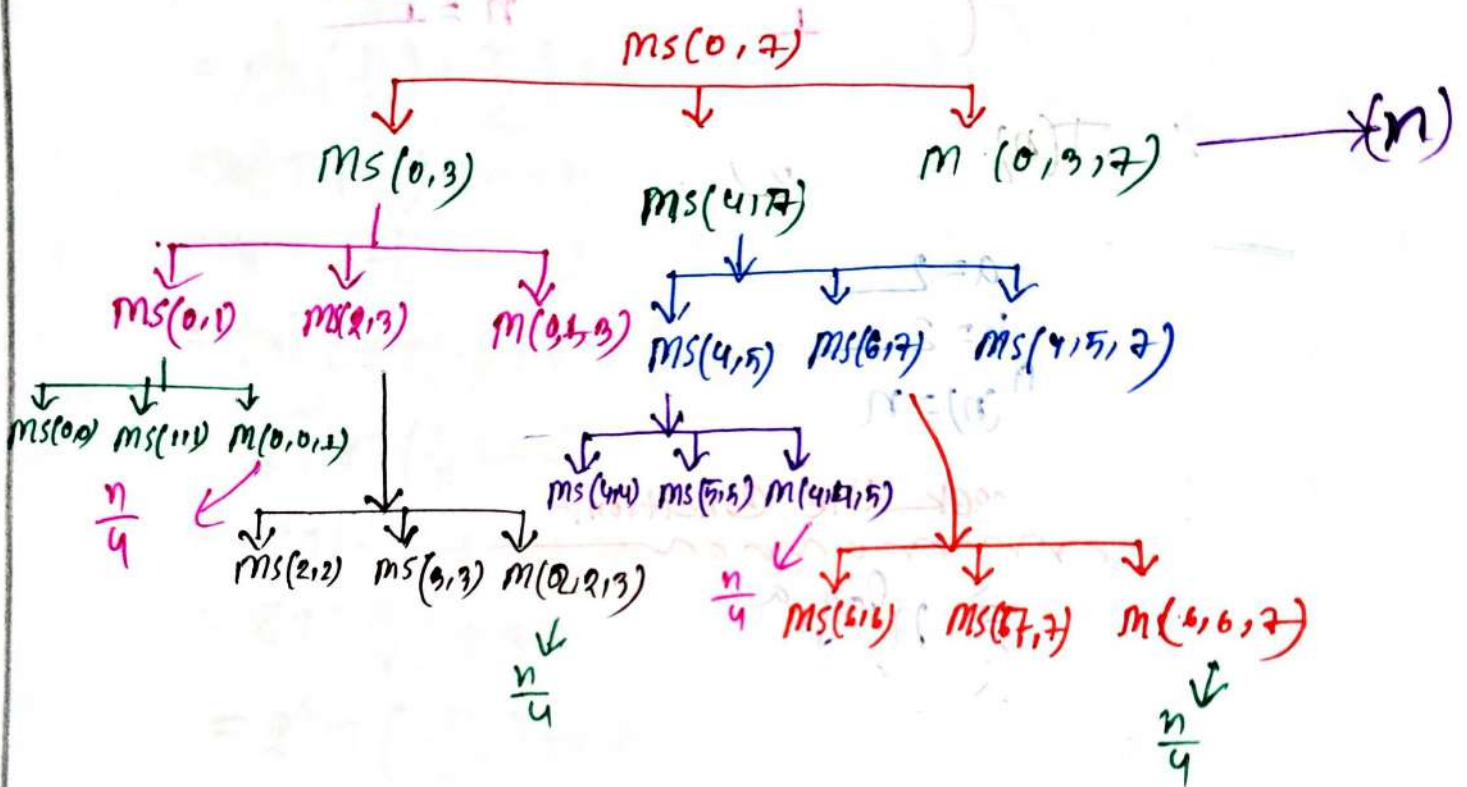
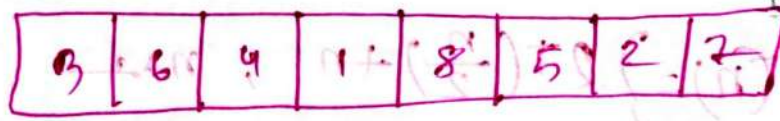
$$\therefore f(n) = n^{\log_b a}$$

$$\begin{aligned} \text{Then, } T(n) &= \Theta(n^{\log_b a} \cdot \log n) \\ &= \Theta(n \cdot \log n) \end{aligned}$$

Ans:-

Merge Sort Time Complexity

Recursive Tree Method:-



$n = 8$	$n = 2^3$	3 levels	need 3 en
$n = 16$	$n = 2^4$	4 levels	4 en
$n = 32$	$n = 2^5$	5 levels	5 en
$n = 64$	$n = 2^6$	6 levels	6 en
n	$n = 2^k$	$\log_2 n$ levels	$\log_2 n \cdot en$

∴ Time Complexity:-

$$O(c \cdot n \log_2 n)$$

$$= O(n \log n)$$

Time Complexity of Insertion Sort

Complexity Analysis of Insertion Sort

Algorithm is:

InsertionSort(A[1..n]) {

for (i = 2; i ≤ n; i++) {

temp = A[i];

j = i - 1;

while (j > 0 && A[j] > temp) {

A[j+1] = A[j];

j--;

A[j+1] = temp;

}

Execute time Cont
mon

→ n times

c₁

→ (n-1) times

c₂

→ (n-1) times

c₃

Best Case : c₄
Worse Case : c₅

→ n-1 ;

→ $\frac{n(n-1)}{2}$

c₄

→ 0 ;

→ $\frac{n(n-1)}{2}$

c₅

→ 0 ;

→ $\frac{n(n-1)}{2}$

c₆

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

→ 0 ;

→ $\frac{n(n-1)}{2}$

For Best Case:

The total Cost time

$$= c_1 n + c_2 (n-1) + c_3 (n-1) + c_4 (n-1) + c_5 \cdot 0 + c_6 \cdot 0 + c_7 (n-1)$$

$$= c_1 n + c_2 n - c_2 + c_3 n - c_3 + c_4 n - c_4 + c_7 n - c_7$$

$$= n(c_1 + c_2 + c_3 + c_4 + c_7) - (c_2 + c_3 + c_4 + c_7)$$

∴ We can write the equation as:-
 $= an + b$

∴ The time complexity is $= O(n)$

For Worst Case:-

The total cost + time:-

$$= c_1n + c_2(n-1) + c_3(n-1) + c_4\left(\frac{n(n-1)}{2}\right) + c_5\left(\frac{n(n-1)}{2}\right) + c_6\left(\frac{n(n-1)}{2}\right) + c_7n$$

$$\Rightarrow c_1n + c_2(n-1) + c_3(n-1) + (c_4n^2 - c_4n)/2 + (c_5n^2 - c_5n)/2 + (c_6n^2 - c_6n)/2 + c_7n$$

$$\Rightarrow n^2\left(\frac{c_4}{2} + \frac{c_5}{2} + \frac{c_6}{2}\right) + n\left(c_1 + c_2 + c_3 - \frac{c_4}{2} - \frac{c_5}{2} - \frac{c_6}{2}\right) + (-c_2 - c_3 - c_7)$$

We can write the equation as:-
 $= an^2 + bn + c$

∴ The time complexity is $= O(n^2)$

Complexity analysis of
Insertion Sort

Algorithm is

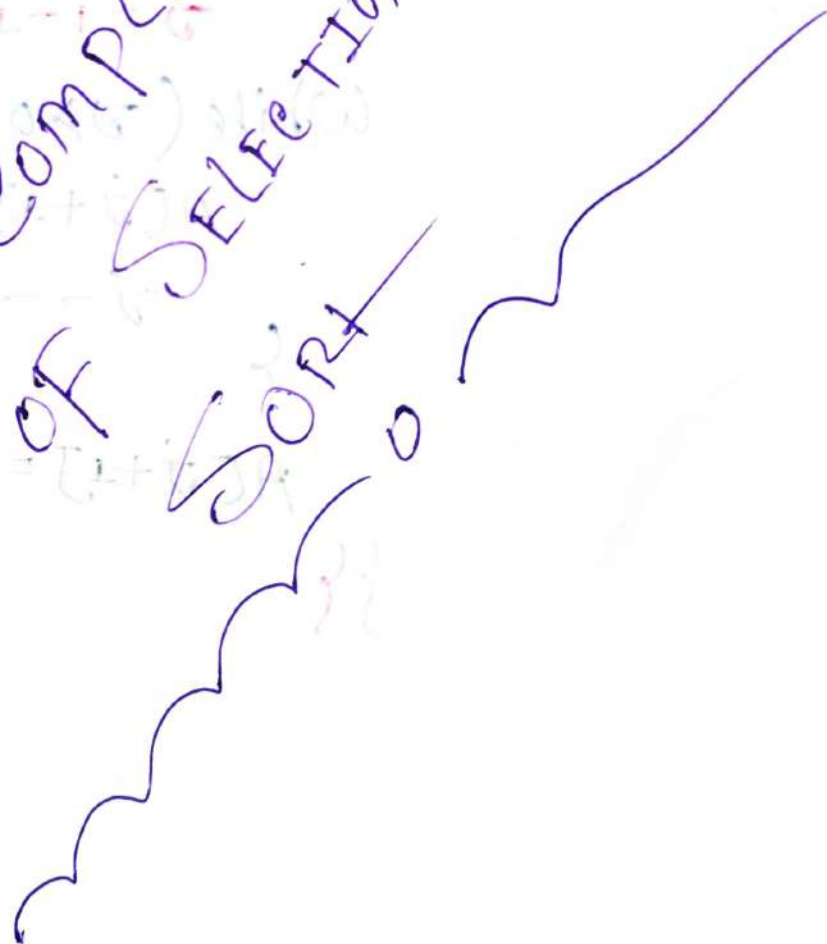
$\{ (n, 234) \}$

TIME COMPLEXITY OF SORT

Insert $n \leftarrow$
Insert $n \leftarrow$
Insert $n \leftarrow$

Insert $n \leftarrow$
Insert $n \leftarrow$
Insert $n \leftarrow$

$\{ (9876543210) \}$



Complexity Analysis of Selection Sort

Algorithm is:-

	Execute time	Cont
selectionSort(A, n)		
for (i = 0; i < n-1; i++)	→ n-1 times	c ₁
int min = i;	→ n-2 times	c ₂
for (j = i+1; j < n; j++)	→ $\frac{n(n-1)}{2}$ times	c ₃
if (A[j] < A[min])	→ $\frac{n(n-1)}{2}$ times	c ₄
min = j;	→ $\frac{n(n-1)}{2}$ times	c ₅
if (min != i)	→ n-2 times	c ₆
swap(A[min], A[i]);	→ n-2 times	c ₇

For Best & Worst

⇒ The total Cont + time:-

$$= c_1(n-1) + c_2(n-2) + c_3\left(\frac{n(n-1)}{2}\right) + c_4\left(\frac{n(n-1)}{2}\right) + c_5\left(\frac{n(n-1)}{2}\right) + c_6(n-2) + c_7(n-2)$$

$$= c_1n - c_1 + c_2n - 2c_2 + (c_3n^2 - c_3n)/2 + (c_4n^2 - c_4n)/2 + (c_5n^2 - c_5n)/2 + c_6n - 2c_6 + c_7n - 2c_7$$

$$= n^2(c_3/2 + c_4/2 + c_5/2) + n(c_1 + c_2 - \frac{c_3}{2} - \frac{c_4}{2} - \frac{c_5}{2} + c_6 + c_7) + (-c_1 - 2c_2 - 2c_6 - 2c_7)$$

We can write the equation as:-

$$= an^2 + bn + c$$

∴ The time complexity is $= O(n^2)$

~~✗~~

Time Complexity of Bubble Sort

Complexity Analysis of Bubble Sorting

⇒ Algorithm is:—

bubbleSort(A[], n)

for (i = 0; i < n-1; i++) → n-1 times

for (j = 0; j < n-1-i; j++) → Best Case n-1 → Worst Case $\frac{n(n-1)}{2}$ times c_2

if (A[j] > A[j+1]) → n-1 → $\frac{n(n-1)}{2}$ times c_3

Swap (A[j], A[j+1]); → 0 → $\frac{n(n-1)}{2}$ times c_4

}}}}

For Best Case:—

total cost time:—

$$= c_1(n-1) + c_2(n-1) + c_3(n-1) + c_4 \times 0 +$$

$$= c_1 n - c_1 + c_2 n - c_2 + c_3 n - c_3$$

$$= n(c_1 + c_2 + c_3) + (-c_1 - c_2 - c_3)$$

we can write the equation as
= an + b

∴ The time complexity = $O(n)$

For Worst Case:—

total cost time:—

$$= c_1(n-1) + c_2\left(\frac{(n-1)n}{2}\right) + c_3\left(\frac{n(n-1)}{2}\right) + c_4\left(\frac{n(n-1)}{2}\right)$$

$$= n^2\left(\frac{c_2}{2} + \frac{c_3}{2} + \frac{c_4}{2}\right) + n\left(c_1 - \frac{c_2}{n} - \frac{c_3}{n} - \frac{c_4}{n}\right) + (-c_1)$$

⇒ we can write the equation as:
= $an^2 + bn + c$

∴ Time Complexity = $O(n^2)$

Time Complexity of Binary Search!

Complexity Analysis of Binary Search

Algorithm is:

binarySearch(A[], x, low, high) {

if (low > high) {

return false; }

else { mid = (low + high) / 2; }

if (x == A[mid]) {

return mid; }

else if (x > A[mid]) {

return binarySearch(A[], x, mid + 1, high);

else {

return binarySearch(A[], x, low, mid - 1);

}

→ O_1

→ $T(1)$

→ $T(\frac{n}{2})$

$$\therefore T(n) = \begin{cases} T(\frac{n}{2}) + T(1) + O_1 & ; n > 1 \\ 1 & ; n = 1 \end{cases}$$

Using Substitution method:-

we have:-

$$T(n) = T\left(\frac{n}{2}\right) + T(1) + c_1 \quad \text{--- (i)}$$

$$\therefore T\left(\frac{n}{2}\right) = T\left(\frac{n}{4}\right) + T(1) + c_1 \quad \text{--- (ii)}$$

$$\therefore T\left(\frac{n}{4}\right) = T\left(\frac{n}{8}\right) + T(1) + c_1 \quad \text{--- (iii)}$$

Substitute the value of (ii) & (iii) in (i):-

$$T(n) = T\left(\frac{n}{2}\right) + c_1$$

$$= T\left(\frac{n}{4}\right) + 2c_1$$

$$= T\left(\frac{n}{8}\right) + 3c_1$$

$$= T\left(\frac{n}{2^k}\right) + kc_1$$

$$= T\left(\frac{n}{2^k}\right) + kc_1 \quad \text{--- (iv)}$$

Let,

$$\frac{n}{2^k} = 1$$

$$\therefore 2^k = n$$

$$\therefore k = \log_2 n$$

Substitute the value in eq (iv)

$$T(n) = T(1) + \log_2 n \cdot c_1$$

$$= 1 + \log_2 n \cdot c_1$$

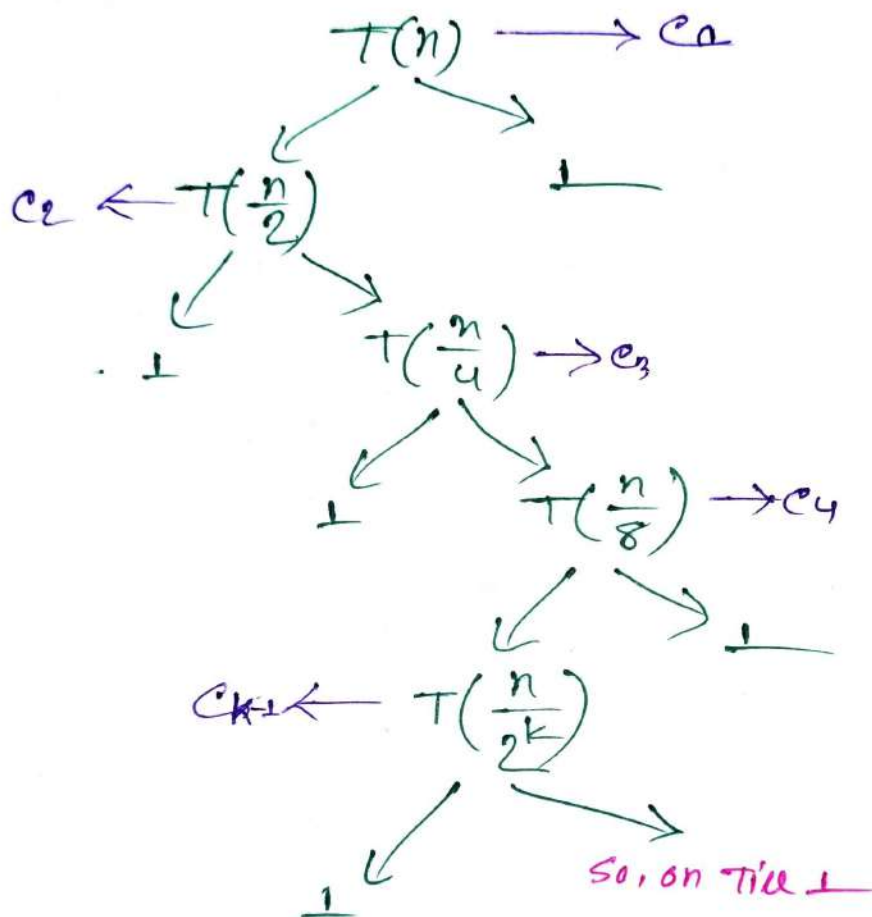
$\therefore$ The complexity is $= O(\log n)$

Ans:-

Using Recursive Tree method:-

$\Rightarrow$ we know:-

$$T(n) = \begin{cases} T(\frac{n}{2}) + c & ; n > 1 \\ 1 & ; n = 1 \end{cases}$$



$\therefore$ Total cost of the tree:-

$$T(n) = c_k$$

Let, $\frac{n}{2^k} = 1$

$$\therefore n = 2^k$$

$$\therefore k = \log_2 n$$

$$\therefore T(n) = c \log_2 n$$

$$\text{Time Complexity} = O(\log n)$$

Linear Search Complexity Analysis

$$T(n) = O(n)$$

or

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

$$T(n) = O(n)$$

Complexity Analysis of Binary Search:-

⇒ Algorithm is:-

LinearSearch (A[1], n)

	<u>Execute time:-</u>	<u>Cost</u>
for (i=0; i < n; i++)	→ n times	c_1
if (a[i] == k)	→ 1 times (Best case) → n times (Worst case)	c_2
print("found");	→ 1 times →	c_3
{		
else { print("Not found"); }	→ 0 times →	$n \cdot \text{times}$
}		

For Best Case:-

total cost time:-

$$= 1 \cdot c_1 + 1 \cdot c_2$$

$$\therefore \text{Time Complexity} = O(1)$$

For Worst Case:-

total cost time:-

$$= n c_1 + n c_2 + n c_3$$

$$= n (c_1 + c_2 + c_3)$$

Can be written as:-

$$= an + o$$

$$\therefore \text{The time complexity} = O(n)$$

Time Complexity of Quick Sort

Complexity Analysis of Quick Sort

⇒ Algorithm is:-

QuickSort(A[], li, mi) {

if (li < mi) {

loc = partition(A[], li, mi); $\xrightarrow{\text{Best Case}} ne \rightarrow \xrightarrow{\text{Worst Case}} ne_1$

QuickSort(A[], li, loc-1); $\rightarrow T(\frac{n}{2}) \rightarrow T(1)$

QuickSort(A[], loc+1, mi); $\rightarrow T(\frac{n}{2}) \rightarrow T(n-1)$

}

$$\therefore T(n) = \begin{cases} 2T(\frac{n}{2}) + ne & ; n \neq 1 \\ 1 & ; n = 1 \end{cases} \quad \left. \vphantom{\begin{matrix} 2T(\frac{n}{2}) + ne \\ 1 \end{matrix}} \right\} \text{for Best Case}$$

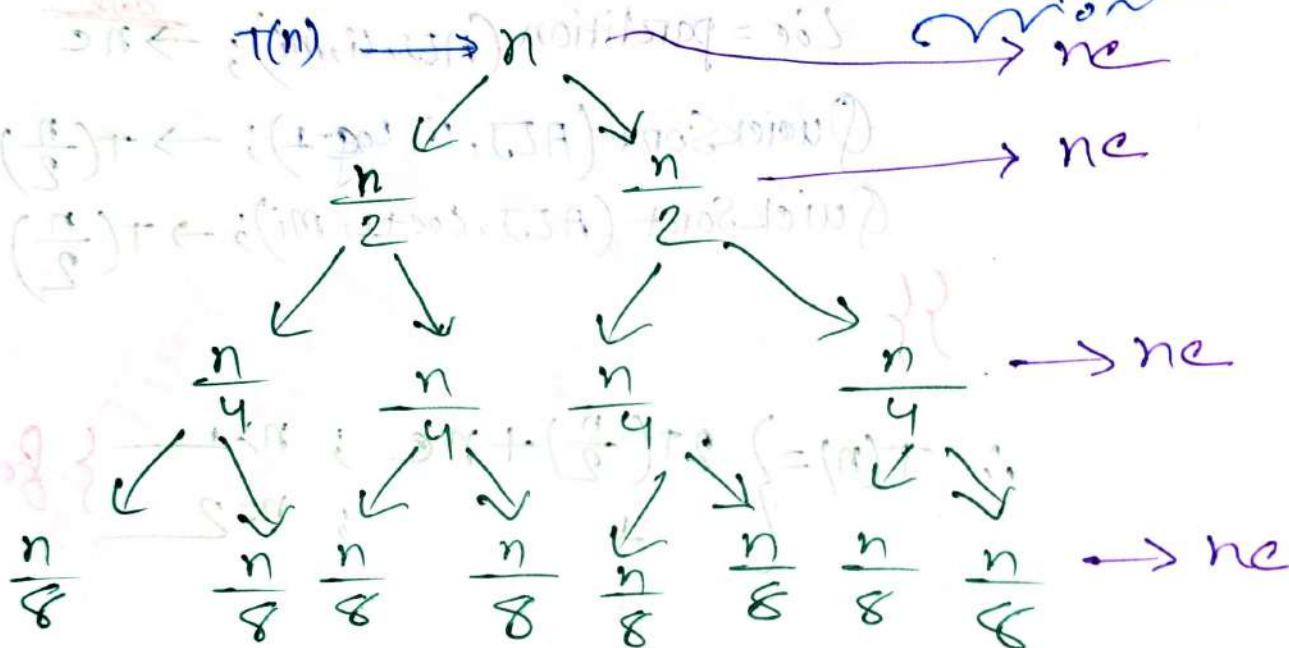
$$\therefore T(n) = \begin{cases} T(n-1) + ne & ; n \neq 1 \\ 1 & ; n = 1 \end{cases} \quad \left. \vphantom{\begin{matrix} T(n-1) + ne \\ 1 \end{matrix}} \right\} \text{for Worst Case.}$$

Best Case Analysis using recursion tree Method:

For Best case:

$$T(n) = \begin{cases} 2T(n/2) + nc & ; n > 1 \\ 1 & ; n = 1 \end{cases}$$

for each level
total n
 $\rightarrow nc$



for last level $\frac{n}{n}, \frac{n}{n}, \frac{n}{n}, \frac{n}{n}, \frac{n}{n}, \frac{n}{n}, \dots \rightarrow nc$

$\therefore$ total level:

$$= \frac{n}{2^0}, \frac{n}{2^1}, \frac{n}{2^2}, \frac{n}{2^3}, \dots, \frac{n}{2^k}$$

$$\therefore \text{total level} = k+1 \quad \text{--- ①}$$

put it in eq ①

$$\therefore \text{total level} = \log_2 n + 1$$

$\therefore$ The total cost

$$\Rightarrow \text{total cost} \times \text{total level} = nc \cdot (\log_2 n + 1)$$

Let, $\frac{n}{2^k} = 1$

$$\Rightarrow 2^k = n$$

$$\therefore k = \log_2 n$$

$\therefore$ the time complexity $T(n) = O(n \cdot \log_2 n + 1)$

Worst case Using
Substitution Method:-

For Worst case:-

$$T(n) = \begin{cases} T(n-1) + n & ; n > 1 \\ 1 & ; n = 1 \end{cases}$$

In given:-

$$T(n) = T(n-1) + n \quad \text{--- (I)}$$

$$T(n-1) = T(n-2) + n \quad \text{--- (II)}$$

$$T(n-2) = T(n-3) + n \quad \text{--- (IV)}$$

Substitute the value in eq (I):-

$$\begin{aligned} \therefore T(n) &= T(n-1) + n \\ &= T(n-2) + 2n \\ &= T(n-3) + 3n \\ &= T(n-k) + kn \quad \text{--- (V)} \end{aligned}$$

Let, $n-k = 1$
 $\therefore k = n$

Now, Substitute the value:-

$$\begin{aligned} \therefore T(n) &= T(1) + n \cdot n \\ &= 1 + n^2 \end{aligned}$$

$\therefore$ The complexity $T(n) = O(n^2)$

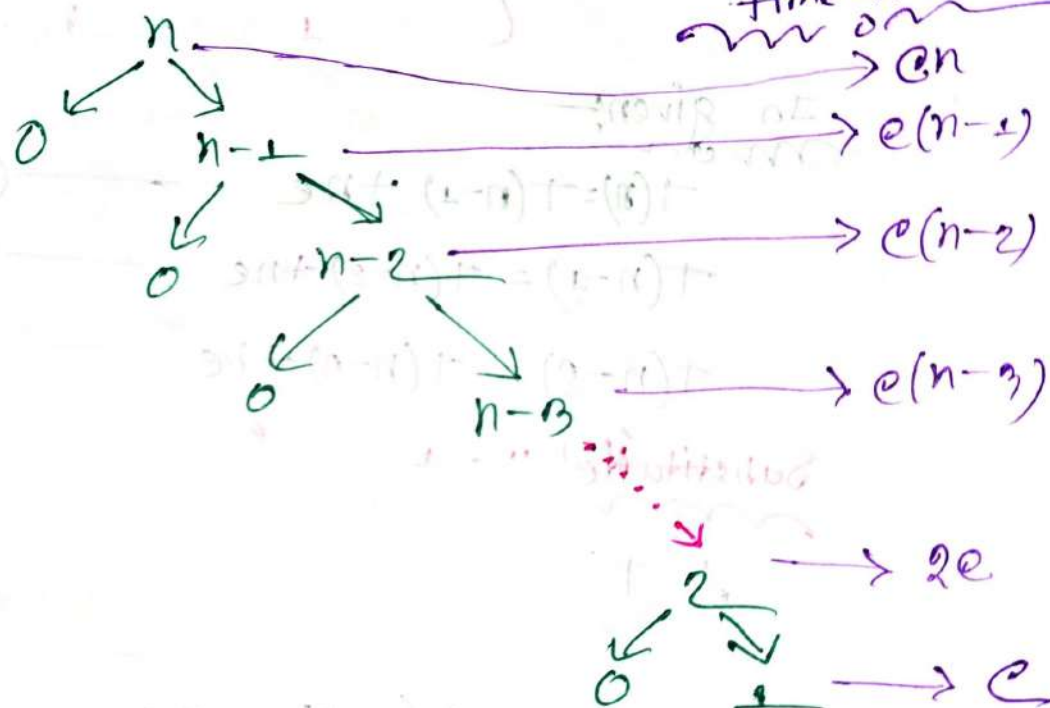
Ans:-

Worst Case Analysis Using Recursion tree Method

The Worst Case

$$T(n) = \begin{cases} T(n-1) + n & ; n > 1 \\ 1 & ; n = 1 \end{cases}$$

Total partition
time at each level
→ cn



∴ Total partition cost:-

$$\begin{aligned} &= cn + c(n-1) + c(n-2) + \dots + 2c + c \\ &= c(n + n-1 + n-2 + \dots + 2 + 1) \\ &= c \left\{ \frac{n(n+1)}{2} \right\} \\ &= (cn^2 + cn) / 2 \end{aligned}$$

∴ The time Complexity is $= O(n^2)$
Ans-

Asymptotic Notations

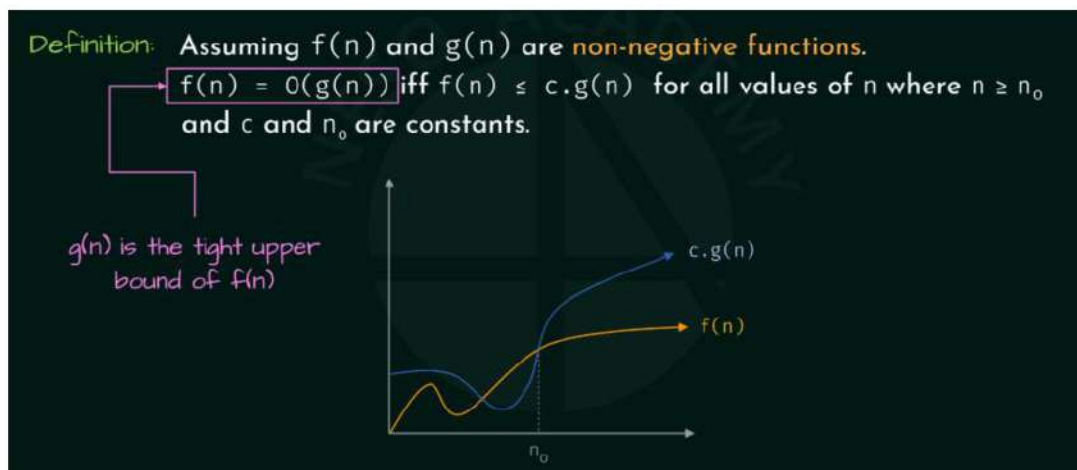
Asymptotic Notations

⇒ Asymptotic Notations are mathematical tools used to analyze the performance of algorithms by understanding how their efficiency changes as the input size grows.

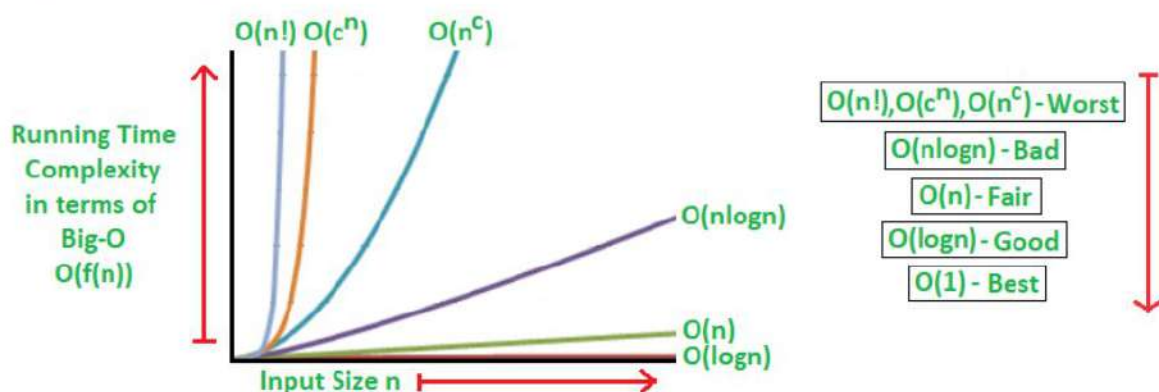
There are mainly three asymptotic notations:

1. Big-O Notation (O-notation)
2. Omega Notation (Ω -notation)
3. Theta Notation (Θ -notation)

Big-O Notation (O-notation)

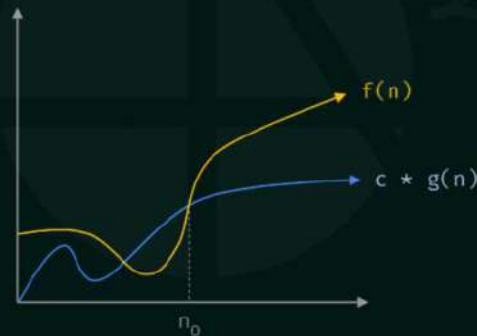


If we plot the most common Big O notation examples, we would have graph like this:



Omega Notation (Ω -notation)

Definition: Assuming $f(n)$ and $g(n)$ are **non-negative functions**.
 $f(n) = \Omega(g(n))$ iff $f(n) \geq c \cdot g(n)$ for some $c > 0$ and for all values of n where $n \geq n_0$ and c and n_0 are constants.



Theta Notation (Θ -Notation)

Definition: Assuming $f(n)$ and $g(n)$ are **non-negative functions**.
 $f(n) = \Theta(g(n))$ iff $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all values of n where $n \geq n_0$ and c_1, c_2 , and n_0 are constants.

$g(n)$ is the asymptotic tight bound of $f(n)$

